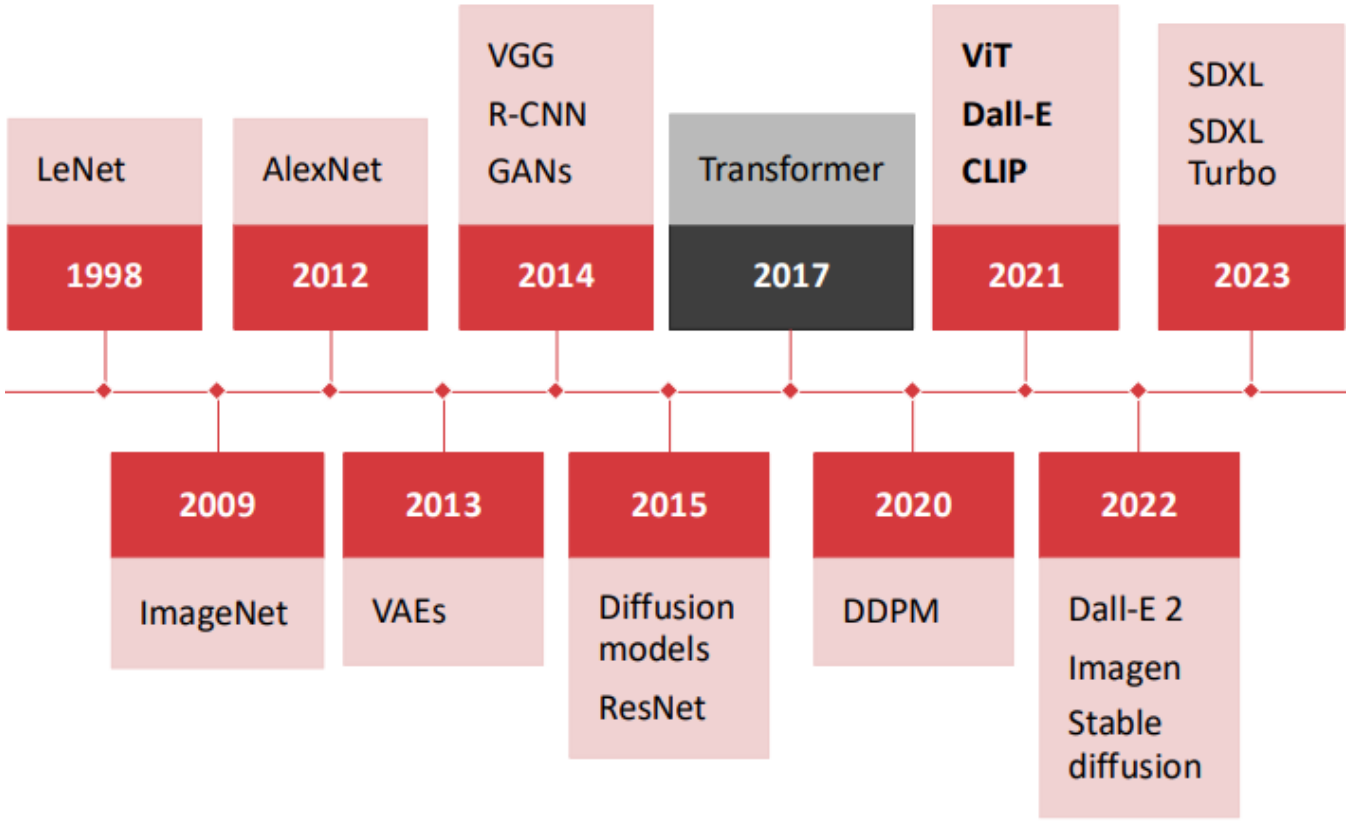
METODE INTELIGENTE DE REZOLVARE A PROBLEMELOR REALE

Laura Dioşan

Vision Transformers

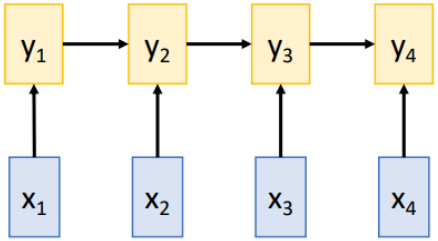
Istoric al modelelor de CV



Procesarea unei secvente

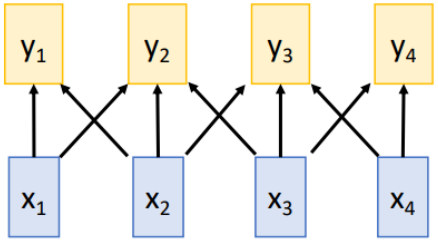
M I R P R

Recurrent Neural Network



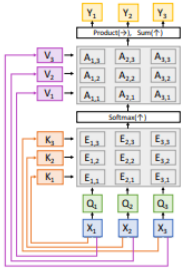
Works on **Ordered Sequences**
(+) Good at long sequences: After one RNN layer, h_T "sees" the whole sequence
(-) Not parallelizable: need to compute hidden states sequentially

1D Convolution



Works on **Multidimensional Grids**
(-) Bad at long sequences: Need to stack many conv layers for outputs to "see" the whole sequence
(+) Highly parallel: Each output can be computed in parallel

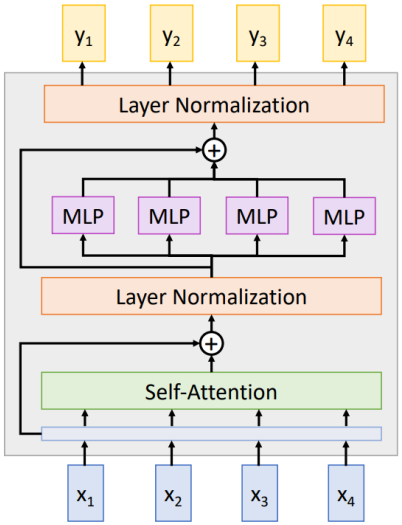
Self-Attention



Works on **Sets of Vectors**
(-) Good at long sequences: after one self-attention layer, each output "sees" all inputs!
(+) Highly parallel: Each output can be computed in parallel
(-) Very memory intensive

Transformer*

- Blocurile unui transformer au
  - Input – vectori (multimi de vectori)
  - Output – vectori (multimi de vectori)
  - Hyper-parametrii:
    - # bloks
    - # heads / block
    - width

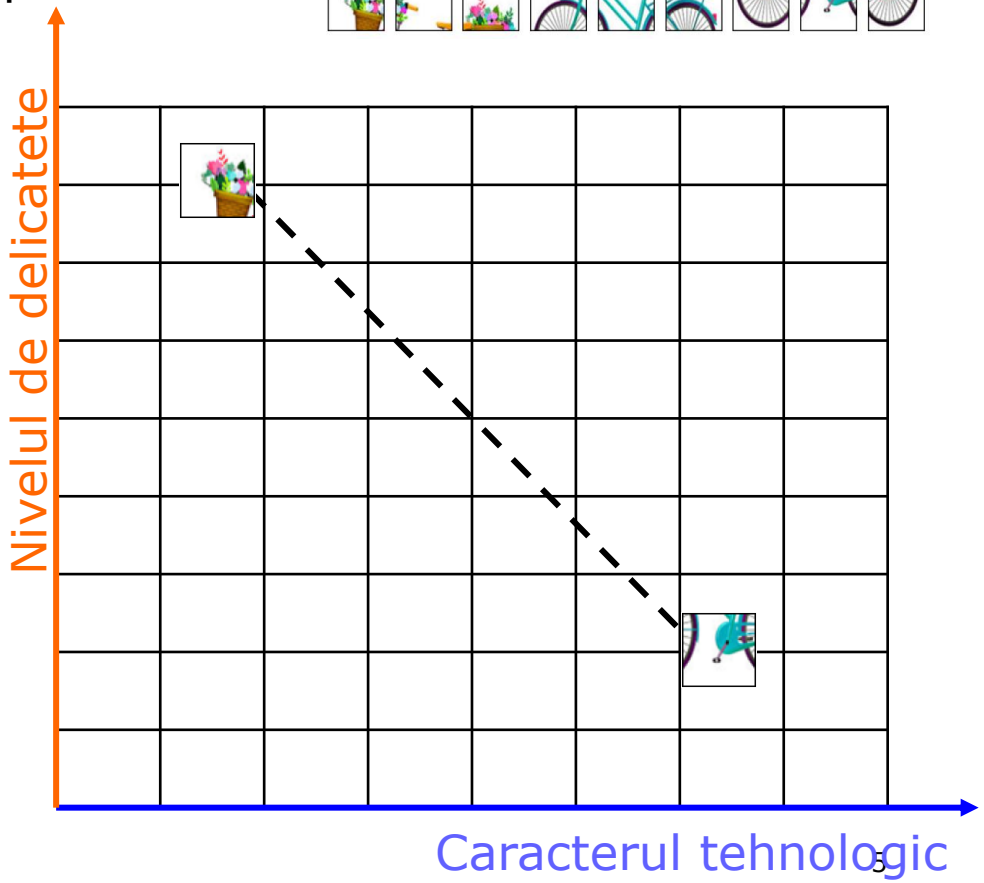


- Vectorii comunica intre ei prin mecanismul de *self-attention* al
  - Unui singur head de procesare
  - A mai multor head-uri de procesare

Vaswani, A. (2017). Attention is all you need. *Advances in Neural Information Processing Systems* link

Mecanismul de “atentie” (attention)

- Remember – embeddings
  - Imagini / patch-uri izolate precum:
  - Presupunem 2 atribute:
    - *caracterul tehnologic*
    - *nivelul de delicatete*



- *Attention*
  - Contextul e ca un magnet!
  - Atrage elementele care

se potrivesc!

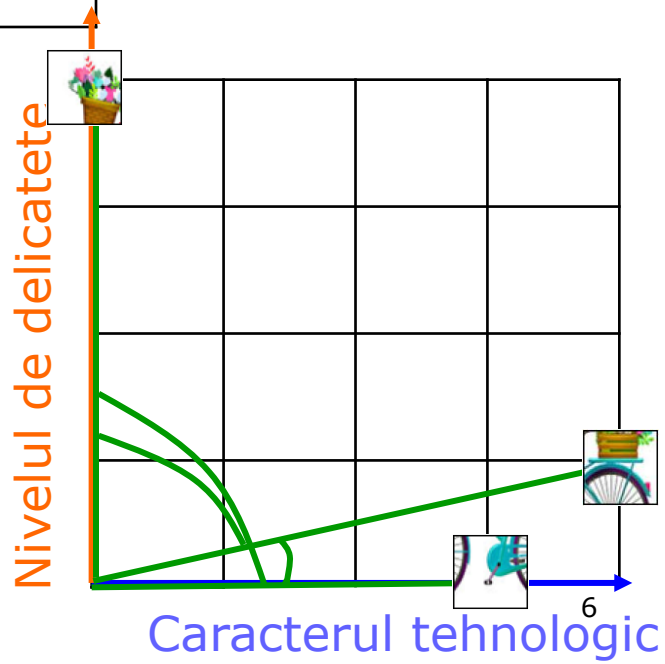
M I R P R

Mecanismul de "atentie" (attention)

- Similaritatea între patch-uri

Cuvântul	*caracterul tehnologic*	*nivelul de delicatete*
	3	0
	4	1
	0	4

- sim ( , ) = 12
  sim([4,1], [3,0]) =
  4*3 + 1*0 = 12
- sim( , ) = 4
  sim([4,1],[0,4]) =
  4*0 + 1*4 = 4
- sim( , ) = 0
  sim([3,0],[0,4]) =
  3*0 + 0*4 = 0



Mecanismul de “atentie” (attention)

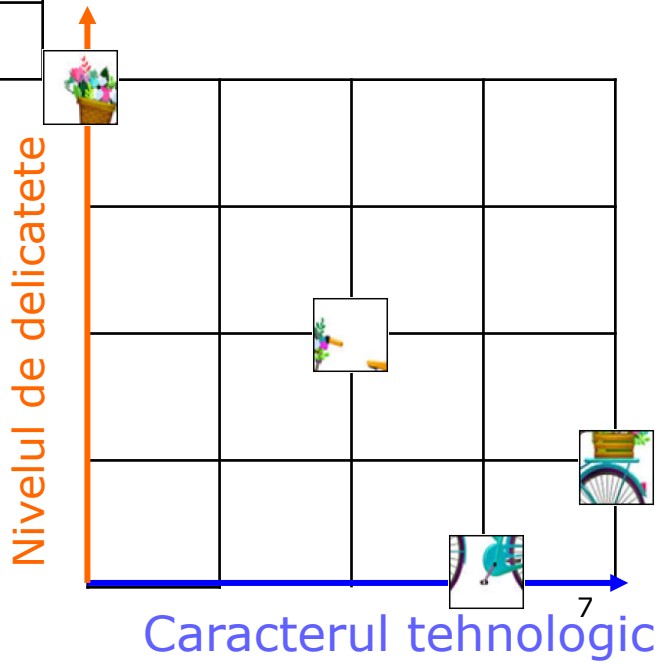
- Contextul patch-urilor – matricea de afinitate

- sim(,) = 1 / √2 = 0.7
- sim(,) =

  sim([2,2], [0,4]) =

  (2*0 + 2*4) / √2 =

  8 /√2 = 5.65



Mecanismul de “atentie” (attention)

- Contextul patch-urilor – matricea de afinitate

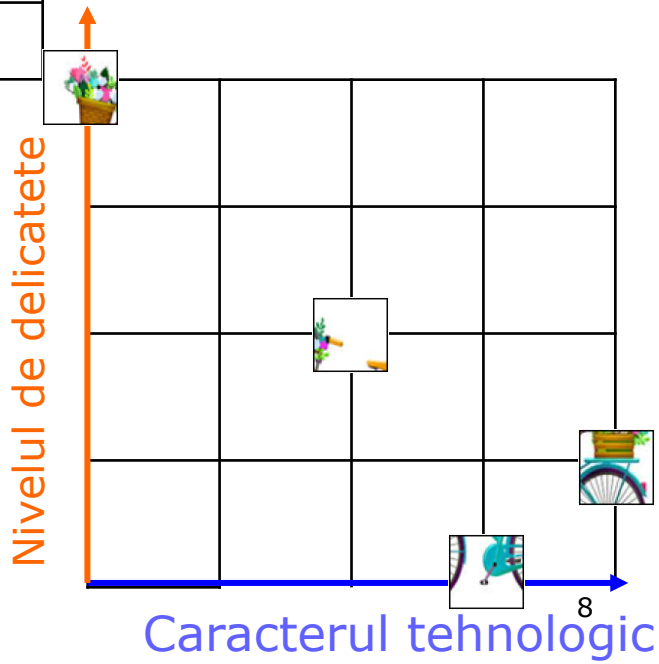
	0.70	5.65
	5.65	0.70

- sim(,) = 1 / √2 = 0.7
- sim(,) =

  sim([2,2], [0,4]) =

  (2*0 + 2*4) / √2 =

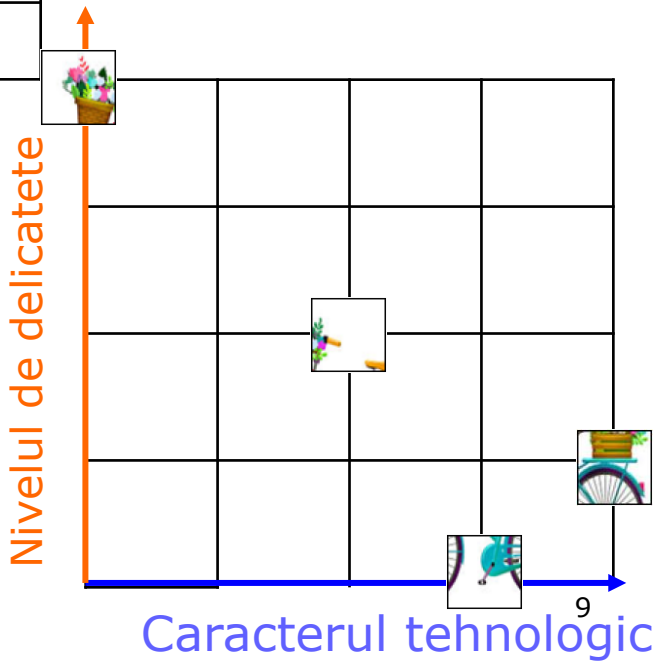
  8 /√2 = 5.65



Mecanismul de “atentie” (attention)

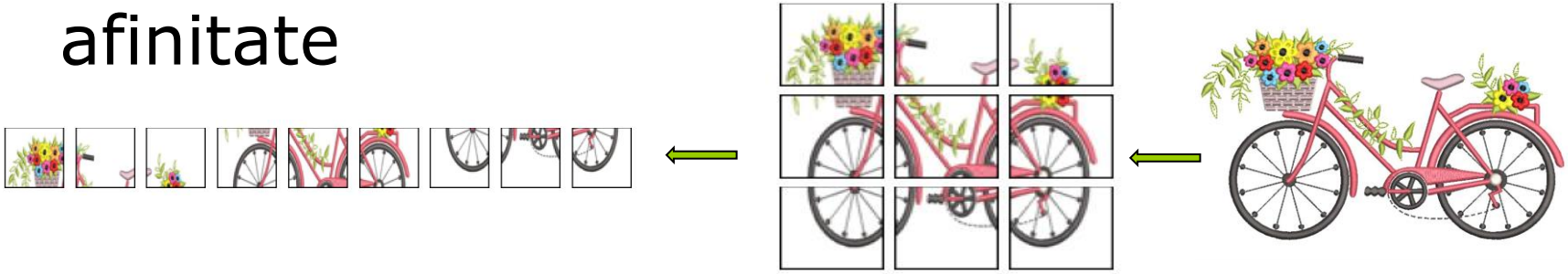
- Contextul patch-urilor – matricea de afinitate 

	0.70	5.65
	5.65	0.70



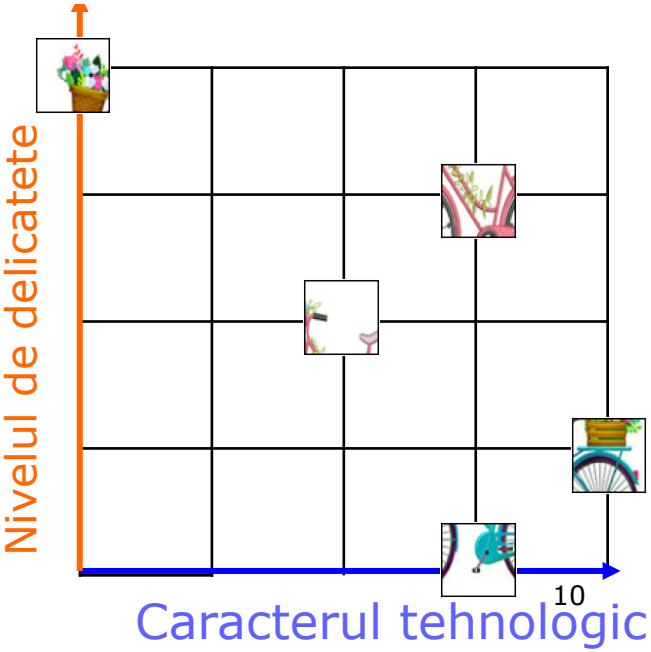
Mecanismul de “atentie” (attention)

- Contextul patch-urilor – matricea de afinitate



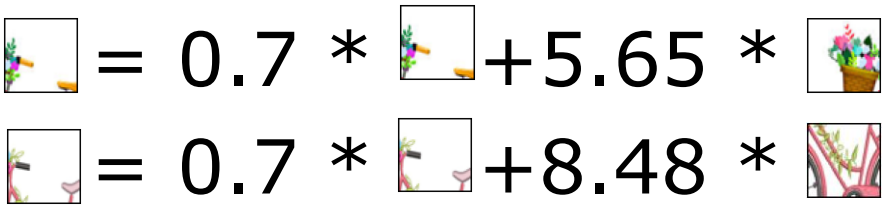
	0.70	8.48
	8.48	0.70

= 0.7 * +8.48 *



Mecanismul de “atentie” (attention)

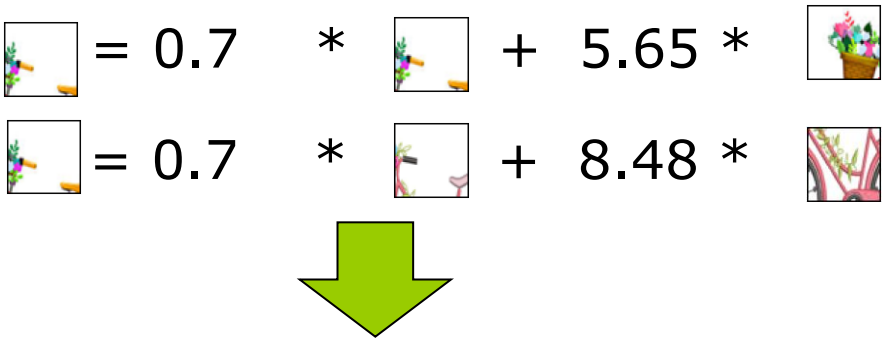
- Contextul patch-urilor – matricea de afinitate
  - 
  - 



Normalizare – de care?

Mecanismul de “atentie” (attention)

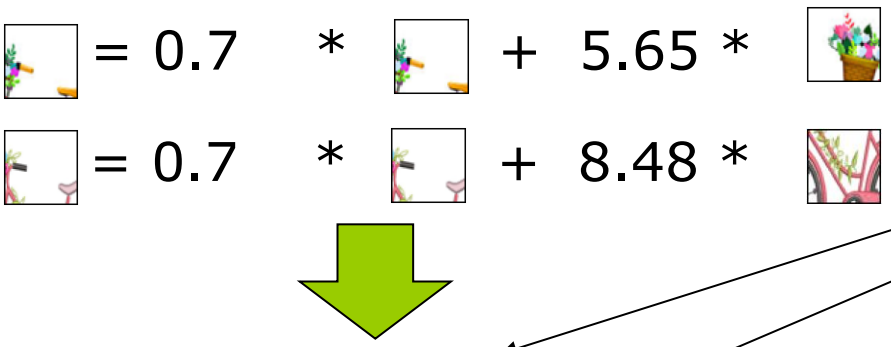
- Contextul patch-urilor – matricea de afinitate
  - 
  - 

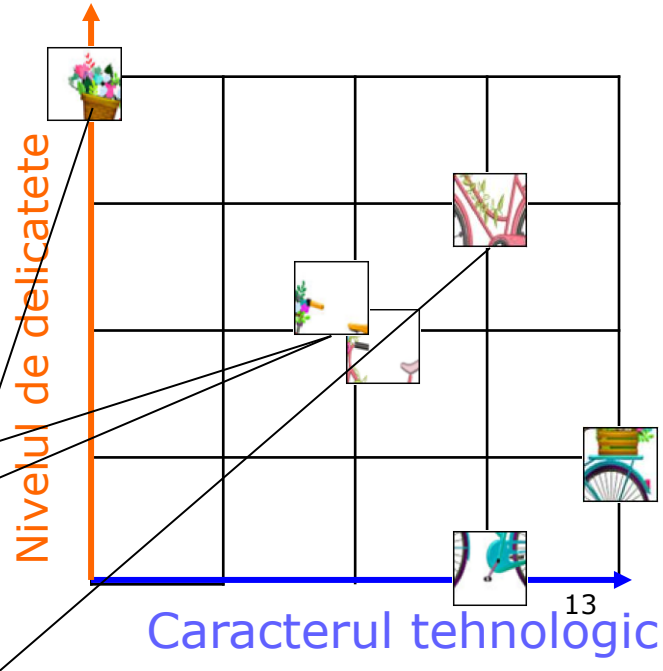


Normalizare – softmax

Mecanismul de “atentie” (attention)

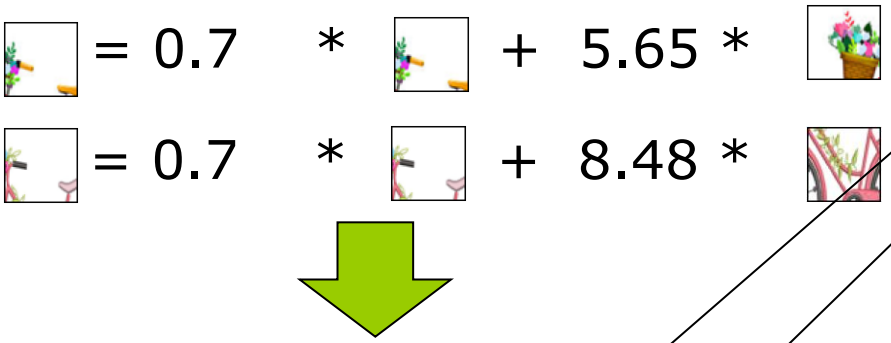
- Contextul patch-urilor – matricea de afinitate

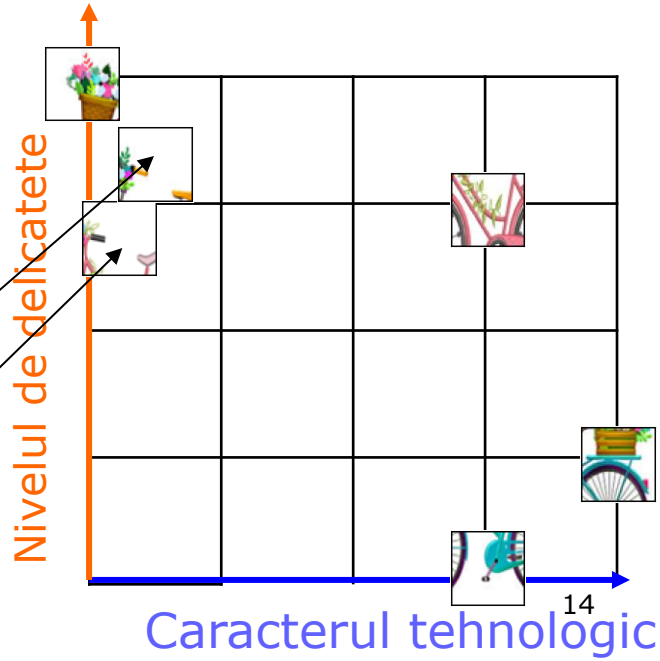




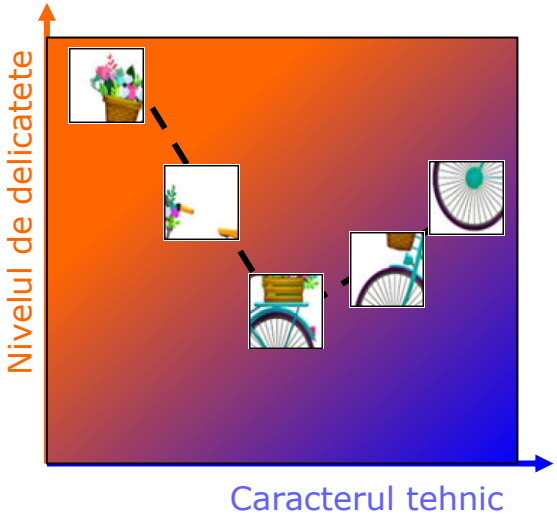
Mecanismul de “atentie” (attention)

- Contextul patch-urilor – matricea de afinitate
  - 
  -

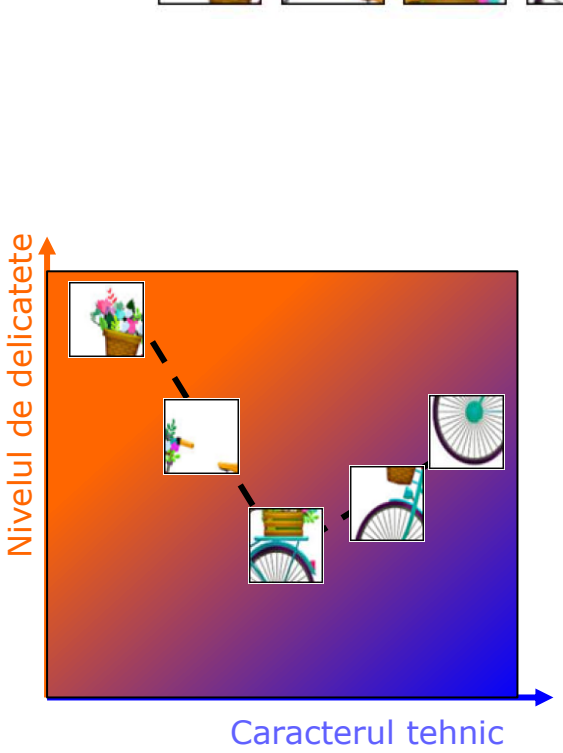


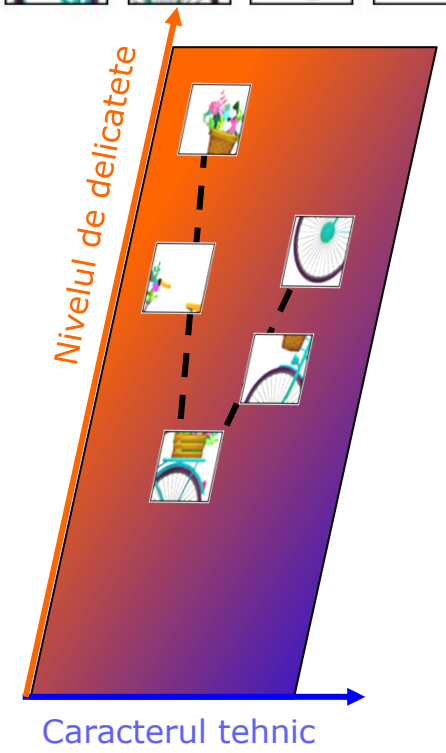


Mecanismul de “atentie” (attention)



Mecanismul de “atentie” (attention)



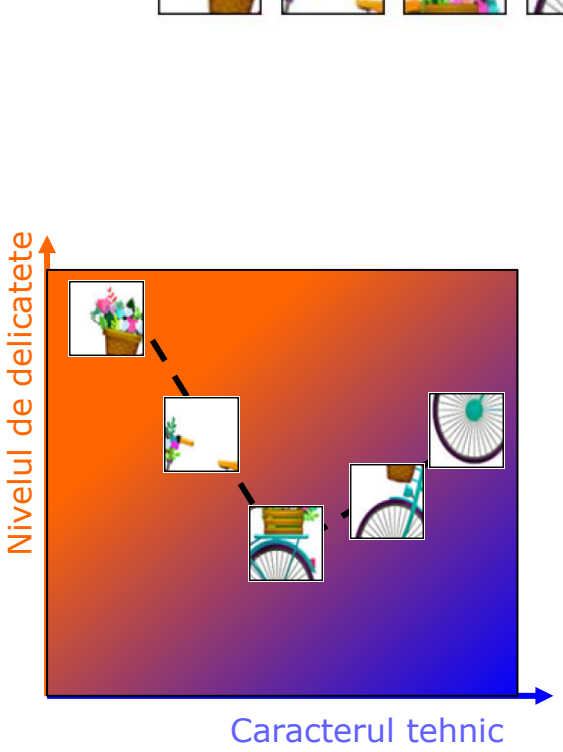


a) reprez1

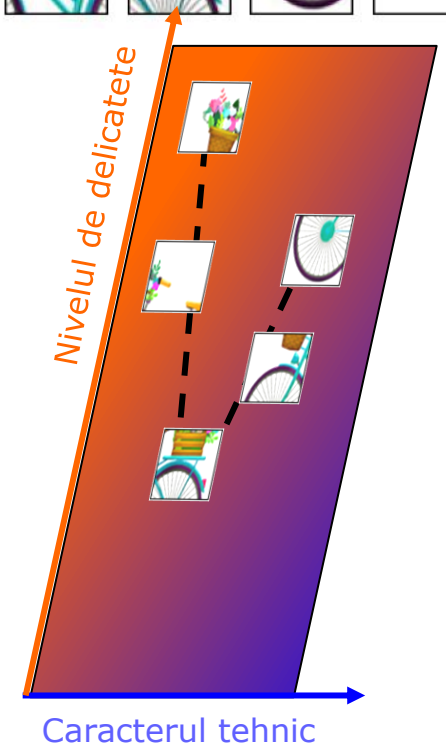
b) reprez2

M
I
R
P
R

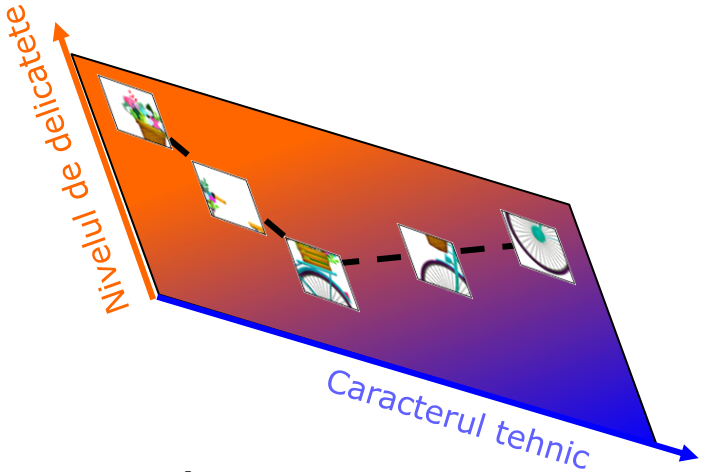
Mecanismul de “atentie” (attention)



a) reprez1

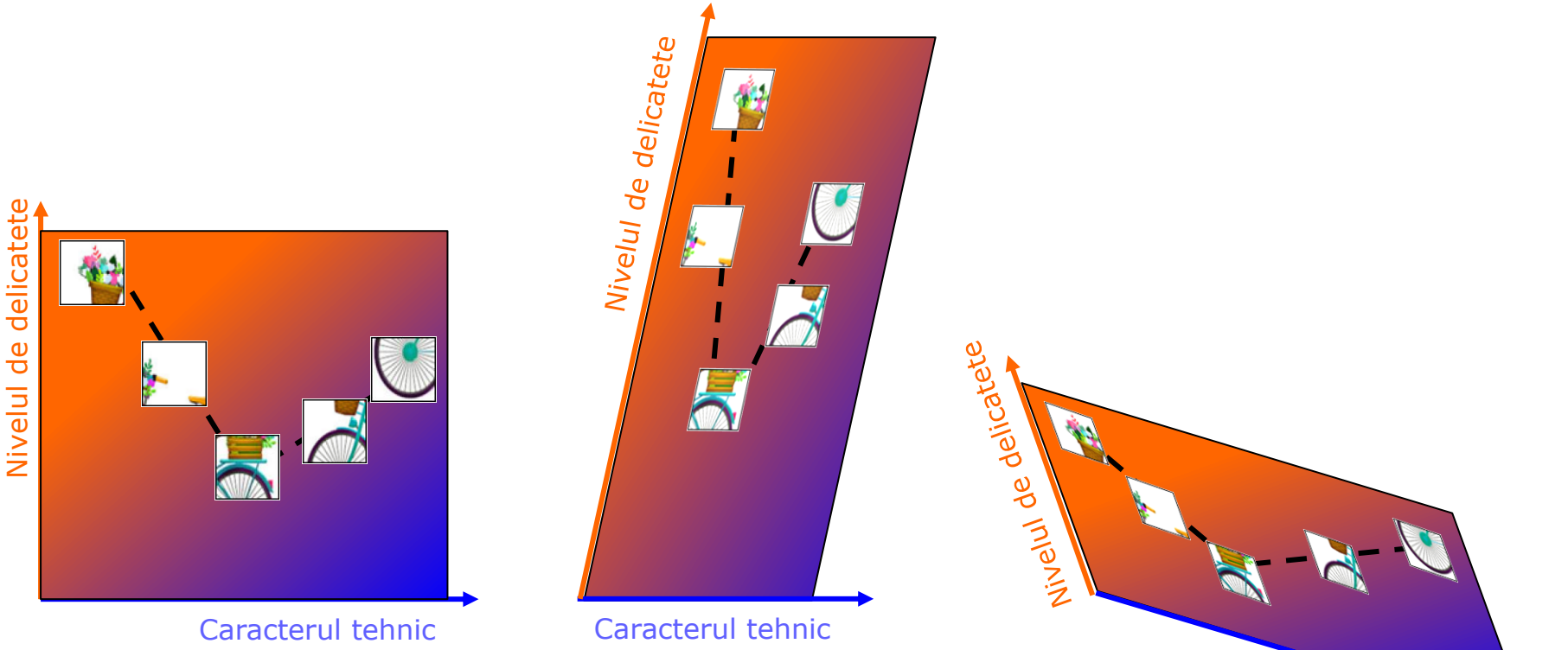


b) reprez2



c) reprez3

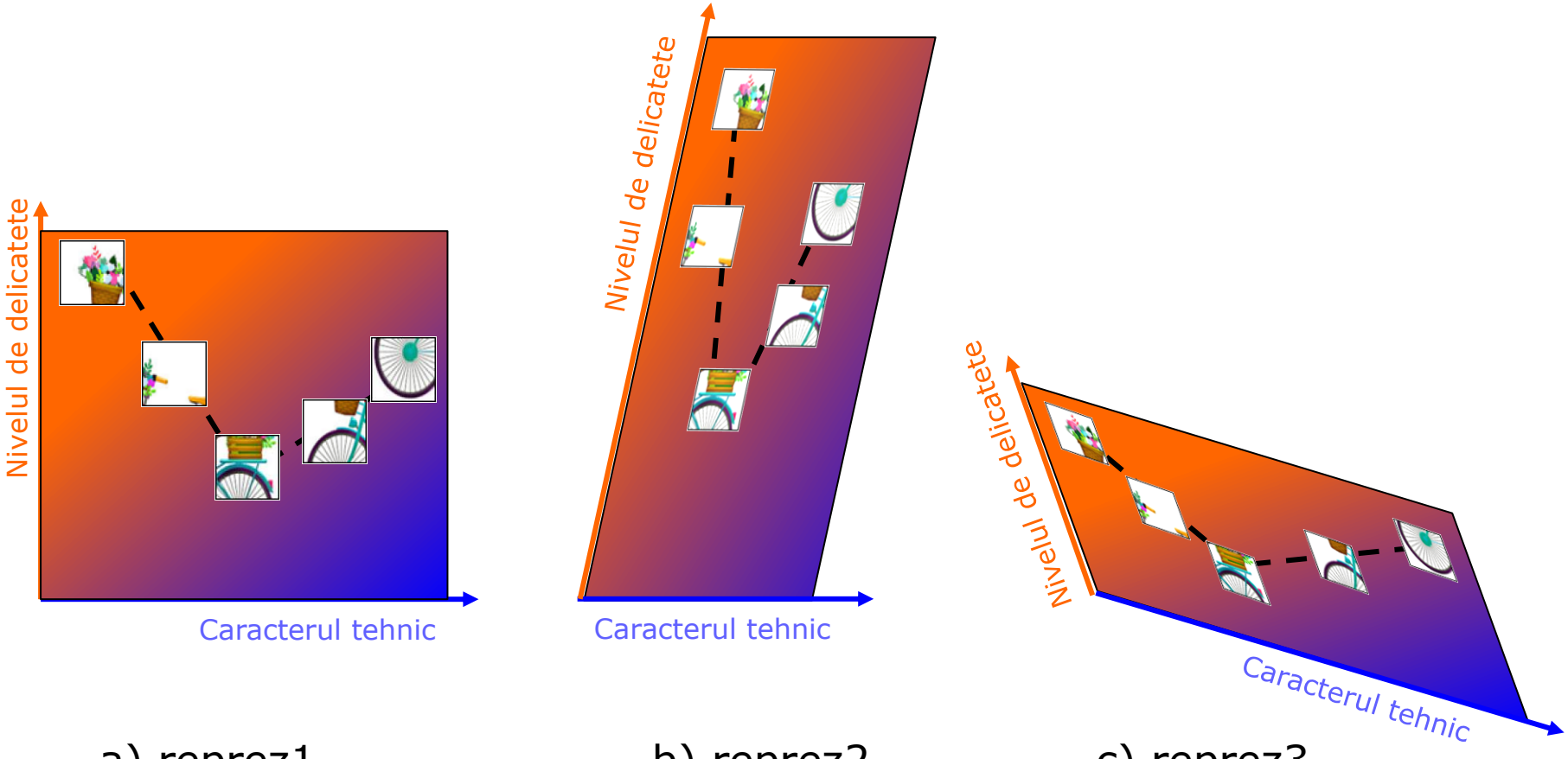
Mecanismul de “atentie” (attention)



a) reprez1 b) reprez2 c) reprez3

Q1: reprez1 = f(reprez2) = g(reprez3) ???
Da!!! Tranformare liniară!!!

Mecanismul de "atentie" (attention)



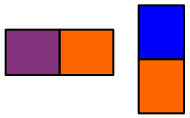
a) reprez1 b) reprez2 c) reprez3

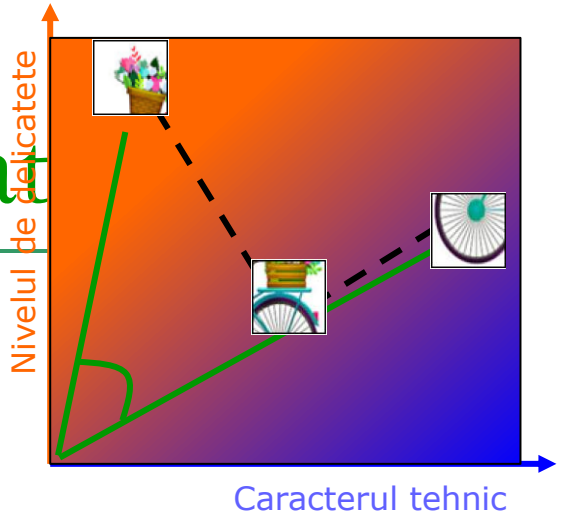
Q2: Care reprezentare e mai bună pentru a stabili similaritatea (deci a face diferența între caracteristicile patch-lui)?

Reprezentarea 1 sau 3!

Mecanismul de “atentie” (a

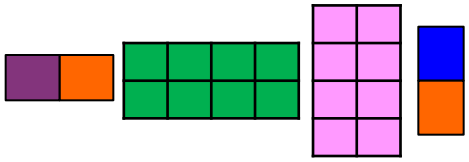
- Sim( , ) =

  sim( , ) = 

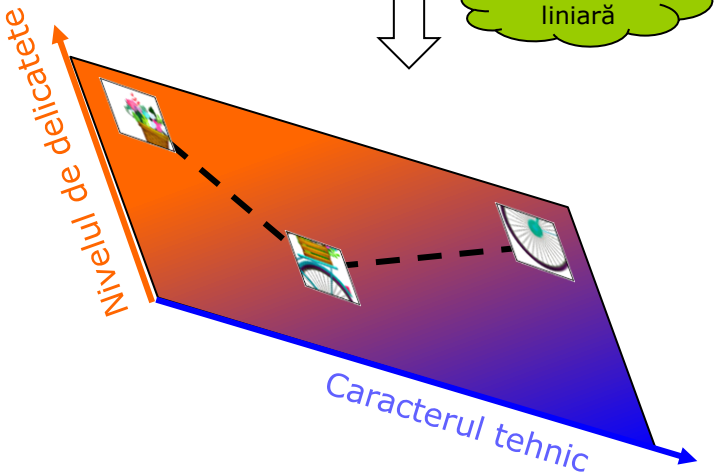


Transformare liniară

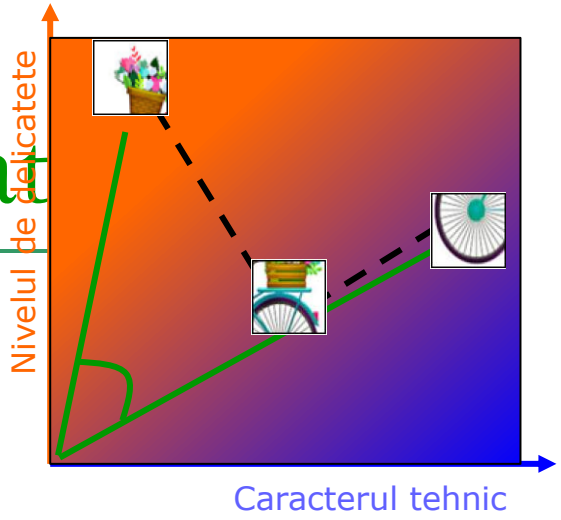
- Sim( , ) =

  sim( , ) = 

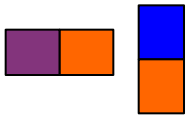
Transformare liniară



Mecanismul de “atentie” (a

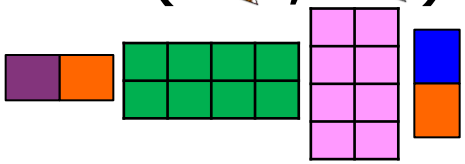


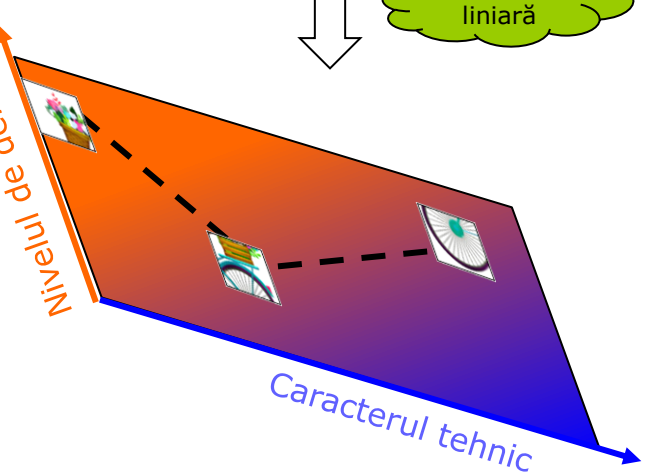
- sim( , ) =

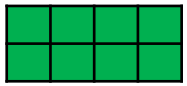
  sim( , ) = 

Transformare liniară

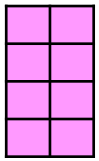
- sim( , ) = sim( , ) =

  sim( , ) = 



K (Key) = 

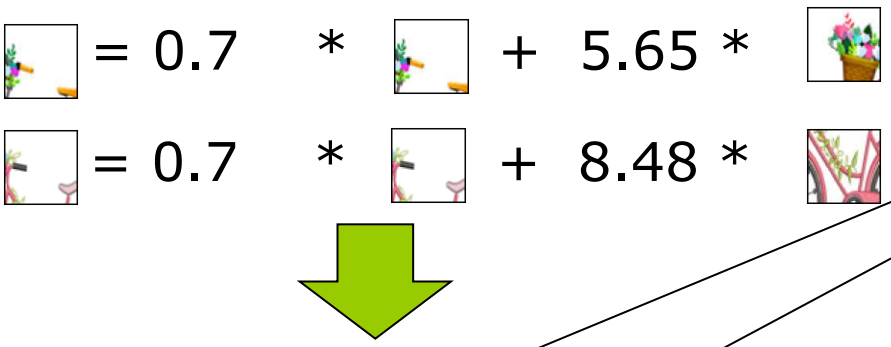
Transformare liniară

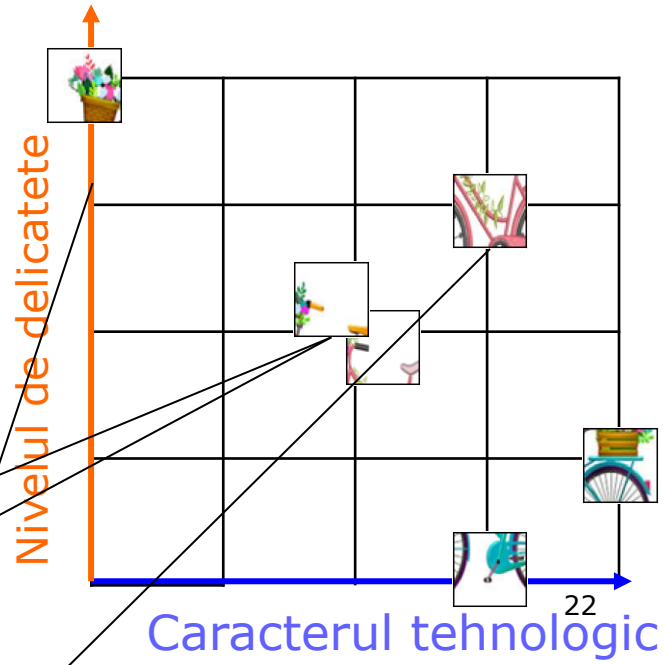
Q (Queries) = 

Valorile optime - ANN

Mecanismul de “atentie” (attention)

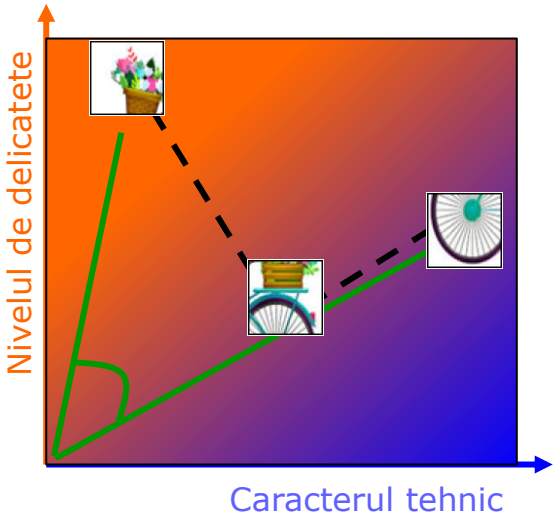
- Contextul patch-urilor – matricea de afinitate
  - 

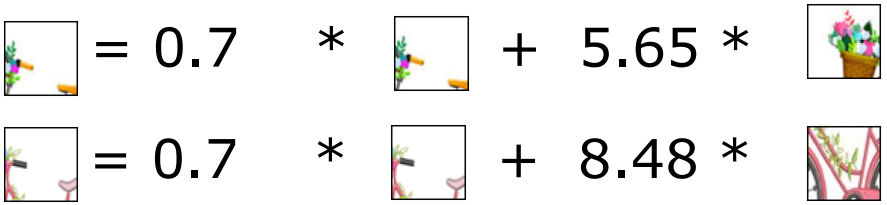




Mecanismul de “atentie” (attention)

- Contextul patch-urilor – matricea de afinitate
  - 
  -





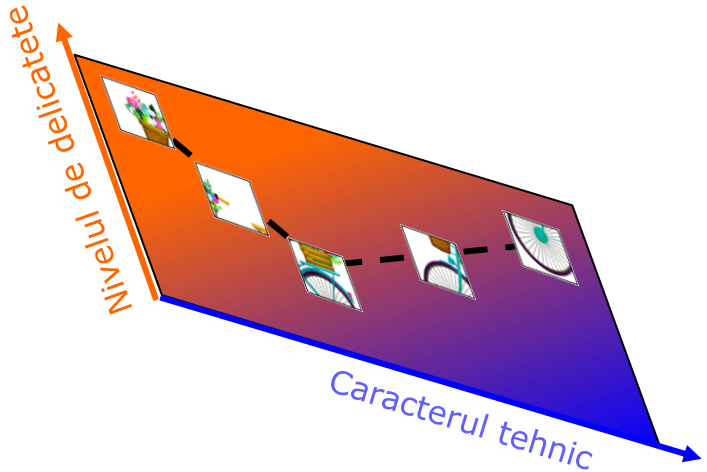

=0.0070 * [2,2] + 0.9929 * [0,4] = [0.1400, 3.9850]

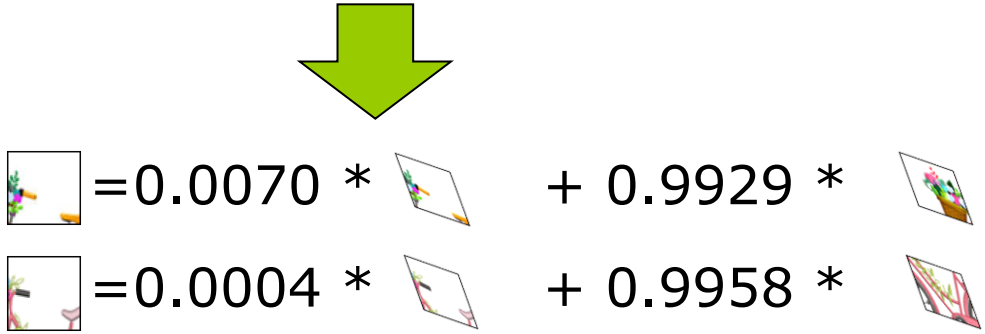

=0.0004 *[2,2] + 0.9958 * [3,3] = [0.0008, 3.9840]

M I R P R

Mecanismul de “atentie” (attention)

- Contextul patch-urilor – matricea de afinitate
  - 
  -

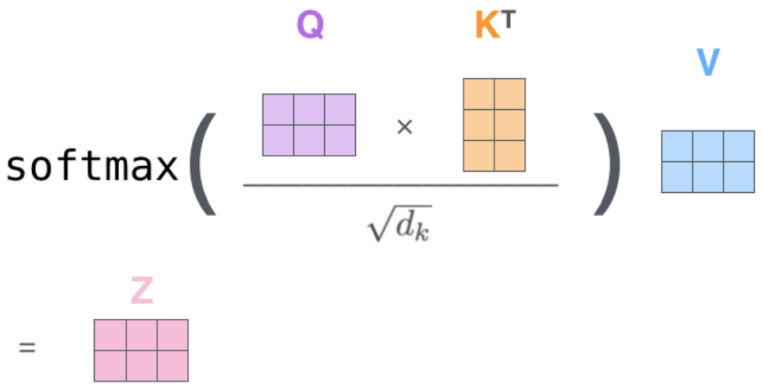




in spatiul proiectiei definite de K si Q

Mecanismul de “atentie” (attention)

- Input – x - vectori de dimensiune (N, #patches x #patches + 1, De)
- Output - vectori de dimensiune (N, #patches x #patches + 1, D)
  - Q = x * W^q
  - K = x * W^k
  - V = x * W^v



Mecanismul de “atentie” (attention)

Input									
representation	x_1								
Queries	q_1								
Keys	k_1								
Values	v_1								
Similarities	$q_1 \cdot k_1$	$q_1 \cdot k_2$	$q_1 \cdot k_3$	$q_1 \cdot k_4$	$q_1 \cdot k_5$	$q_1 \cdot k_6$	$q_1 \cdot k_7$	$q_1 \cdot k_8$	$q_1 \cdot k_9$
Scores s= [s1, s2, .., s9]	$q_1 \cdot k_1 / \sqrt{D}$	$q_1 \cdot k_2 / \sqrt{D}$	$q_1 \cdot k_3 / \sqrt{D}$	$q_1 \cdot k_4 / \sqrt{D}$	$q_1 \cdot k_5 / \sqrt{D}$	$q_1 \cdot k_6 / \sqrt{D}$	$q_1 \cdot k_7 / \sqrt{D}$	$q_1 \cdot k_8 / \sqrt{D}$	$q_1 \cdot k_9 / \sqrt{D}$
w = Softmax(s)									
Weightening the values	$z_1 = w_1 * v_1 + w_2 * v_2 + \ldots + w_9 * v_9$								

Mecanismul de "atentie" (attention)

- Interogari intr-o BD
  - Intput:
    - O multime de recorduri { (key_i, $value_i$), i =1,2, …, m}
    - Un query q
  - Output
    - $value_j$ a.i. q e similar cu key_j
  - Exemplu - text
    - {(Apostol, Mihai), (Baciu, Alina), (Cretu, Gabriel), (Pop, Ionica),(Popa, Madalina), (Popescu, Cosmina)}
    - Q = "Pop"
    - Answer:
      - Ionica – perfect match (e.g. Hamming Distance)
      - Ionica, Madalina, Cosmina – partial match (e.g. Levenshtein Distance), top-k raspunsuri

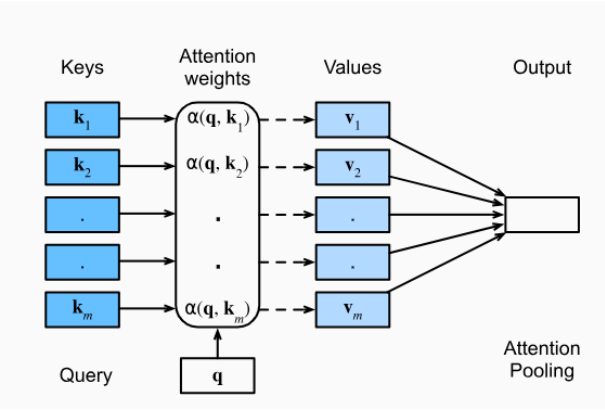
Q=Pop	Apostol	Baciu	Cretu	Pop	Popa	Popescu
Hamming	6	5	5	0	4	7
Levenshtein	6	5	5	0	1	4

Mecanismul de "atentie" (attention)

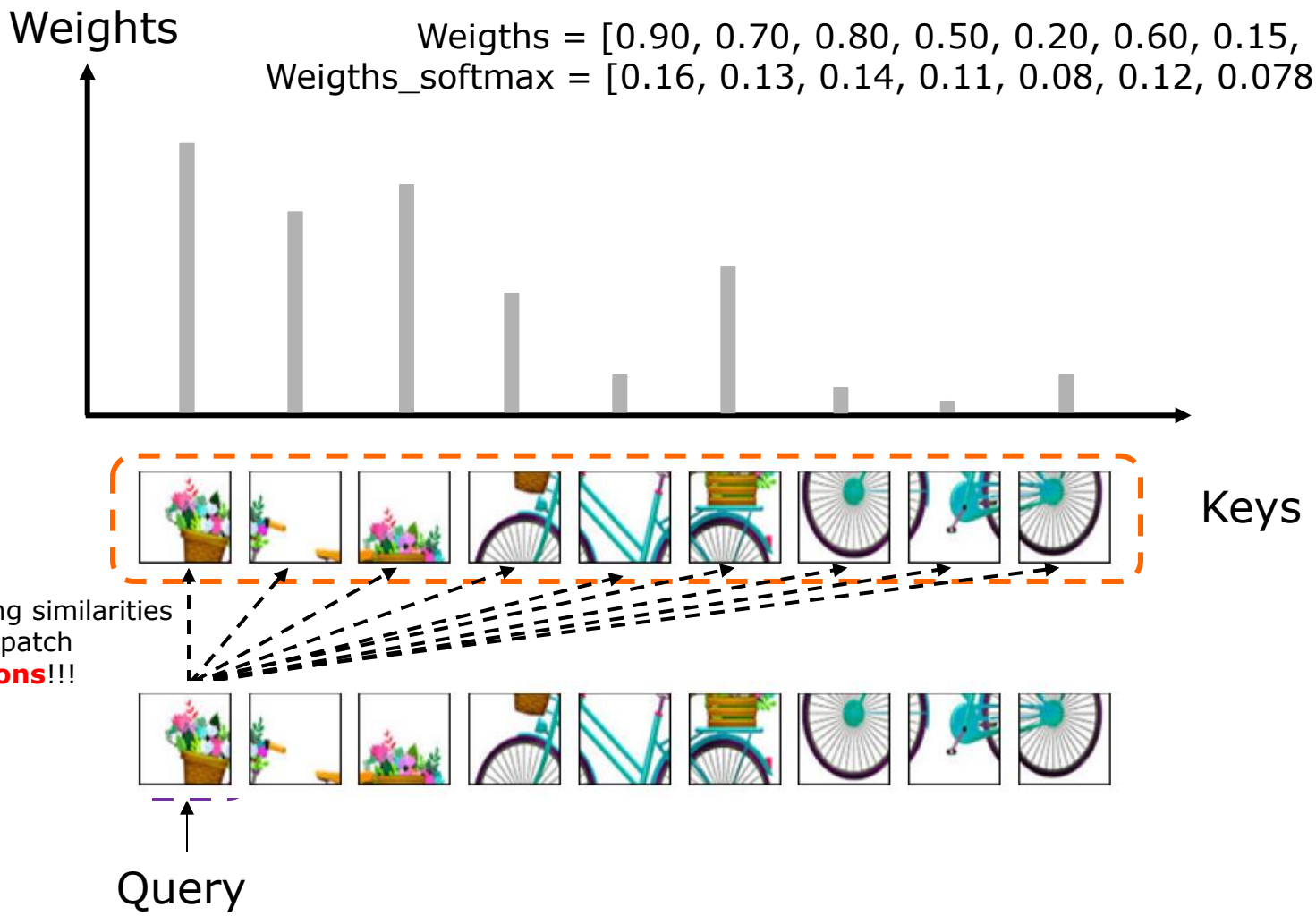
- Interogari intr-o baza de date
  - Exemplu - text
    - DB = {(Apostol, Mihai), (Baciu, Alina), (Cretu, Gabriel), (Pop, Ionica),(Popa, Madalina), (Popescu, Cosmina)}
    - Q = "Pop"
    - Answer:
      - {Ionica} – perfect match (e.g. Hamming Distance)
      - {Ionica, Madalina, Cosmina} – partial match (e.g. Levenshtein Distance), top-k raspunsuri

Q=Pop	Apostol	Baciu	Cretu	Pop	Popa	Popescu
Hamming	6	5	5	0	4	7
Levenshtein	6	5	5	0	1	4

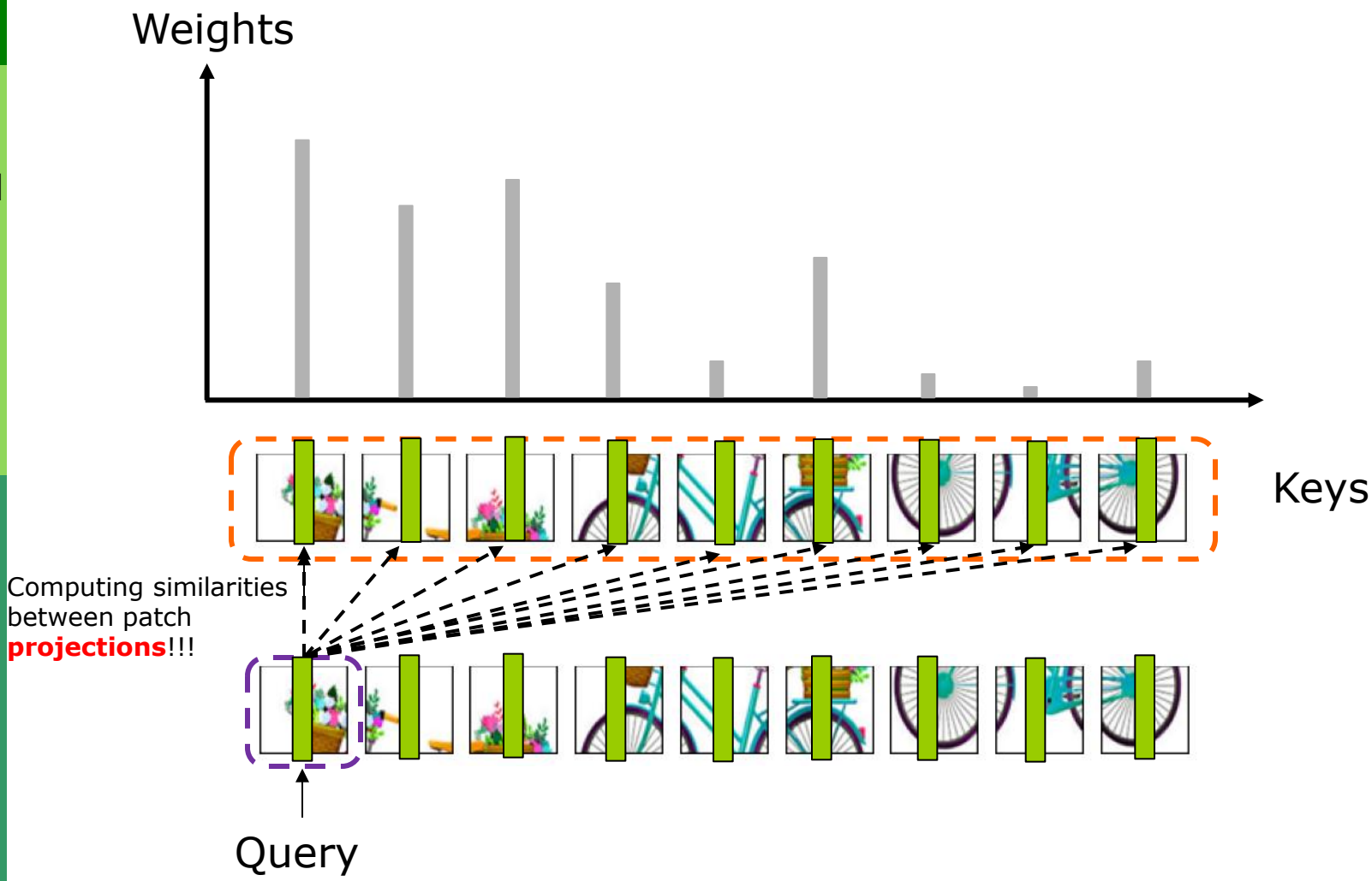
  - Mecanismul de atentie
    - Attention(q,DB) = $\Sigma\ \alpha(q,key_i)*value_i$,
    - $\alpha(q,key_i)$ – attention weights
      - Daca un singur coefficient e 1 si restul 0 → interogari clasice in baze de date
      - Daca toti coeficientii α = 1/m → Average Pooling
      - Pentru a avea coeficienti α ca ponderi (weights)
        - $\alpha'(q,key_i) = \alpha(q,key_i) / \Sigma\ \alpha(q,key_j)$
        - $\alpha'(q,key_i) = \exp(\alpha(q,key_i)) / \Sigma \exp(\alpha(q,key_j))$



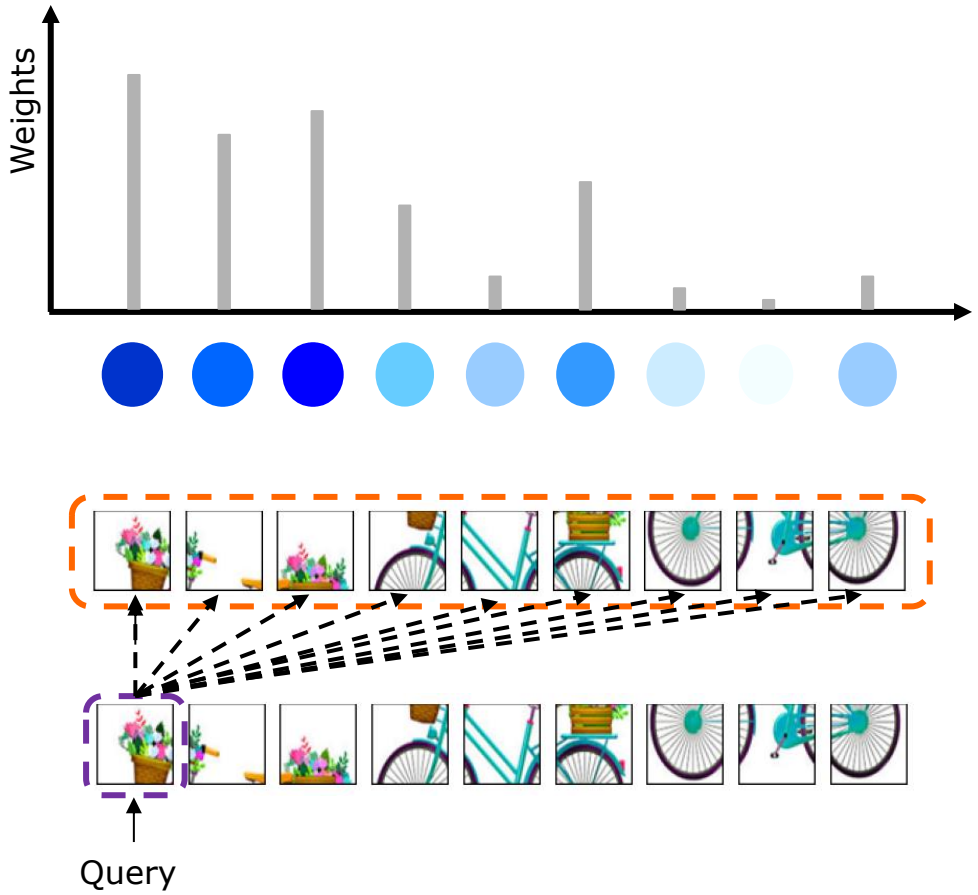
Mecanismul de “atentie” (attention)

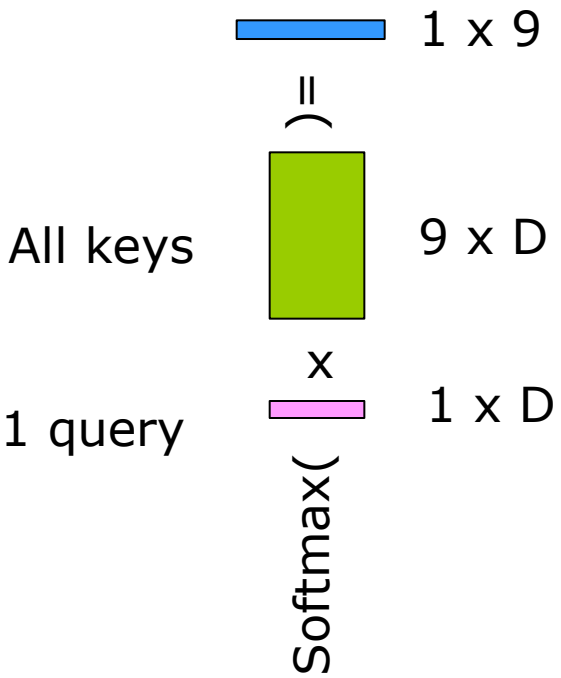


Mecanismul de “atentie” (attention)

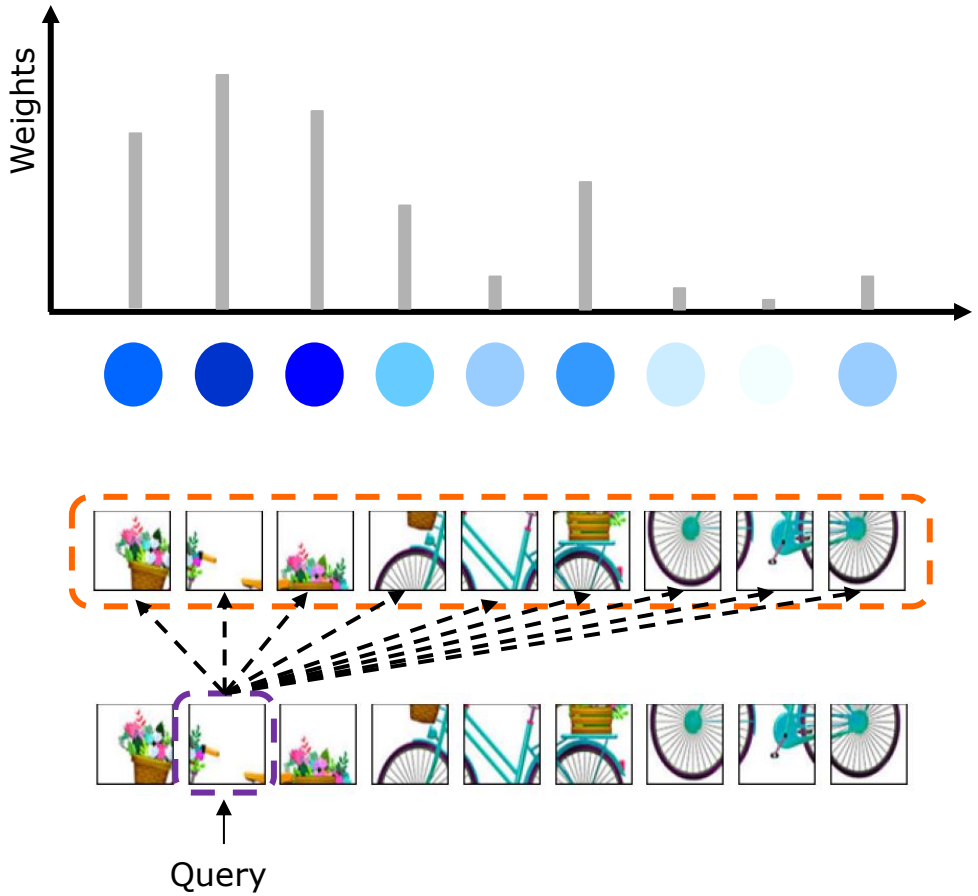


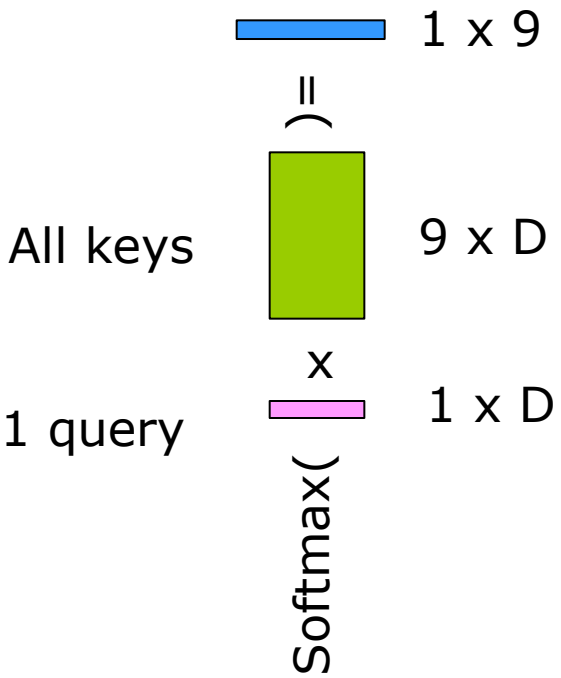
Mecanismul de “atentie” (attention)



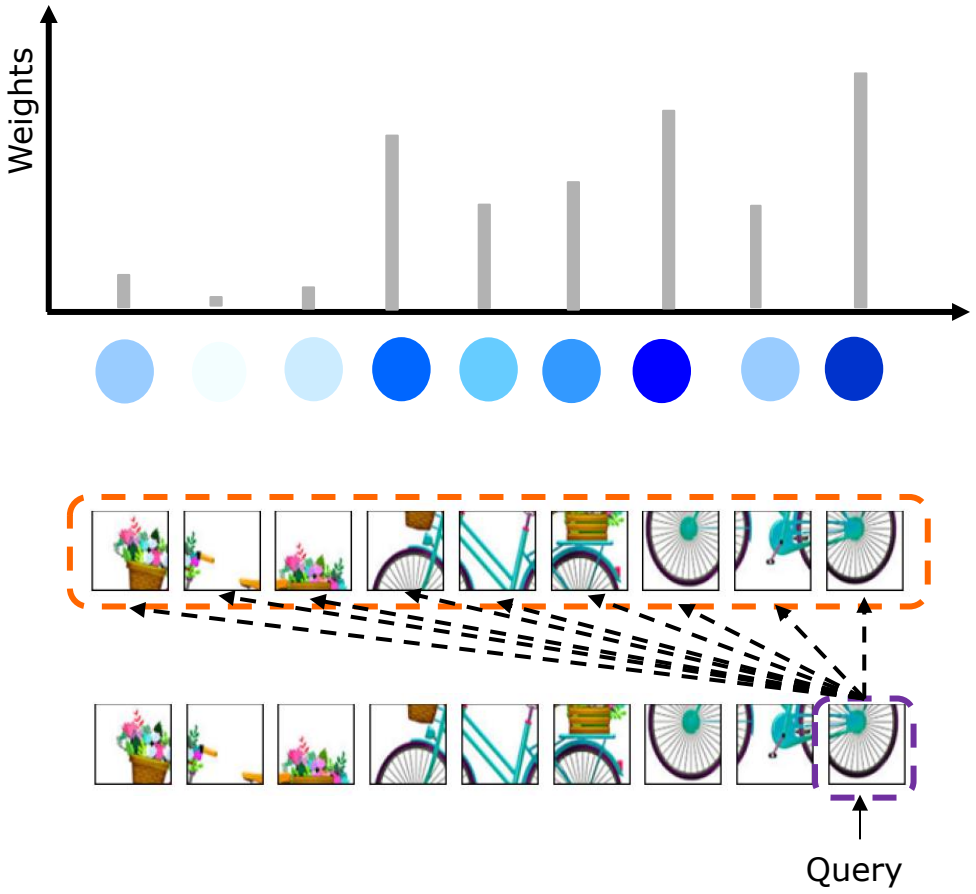


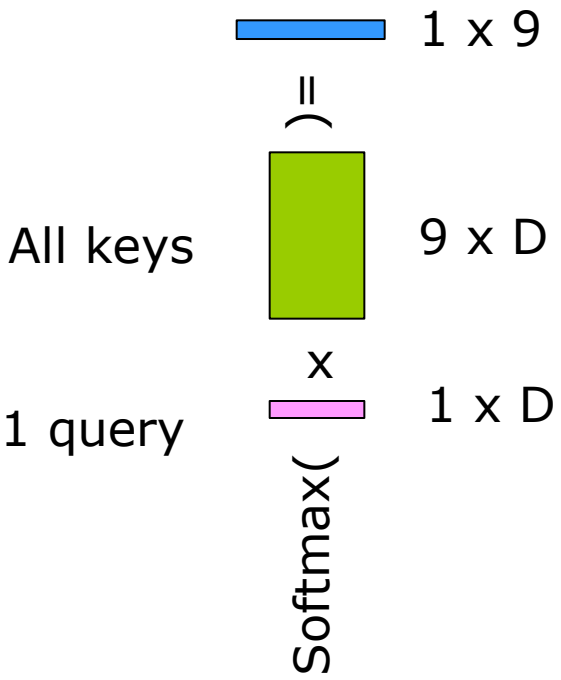
Mecanismul de “atentie” (attention)



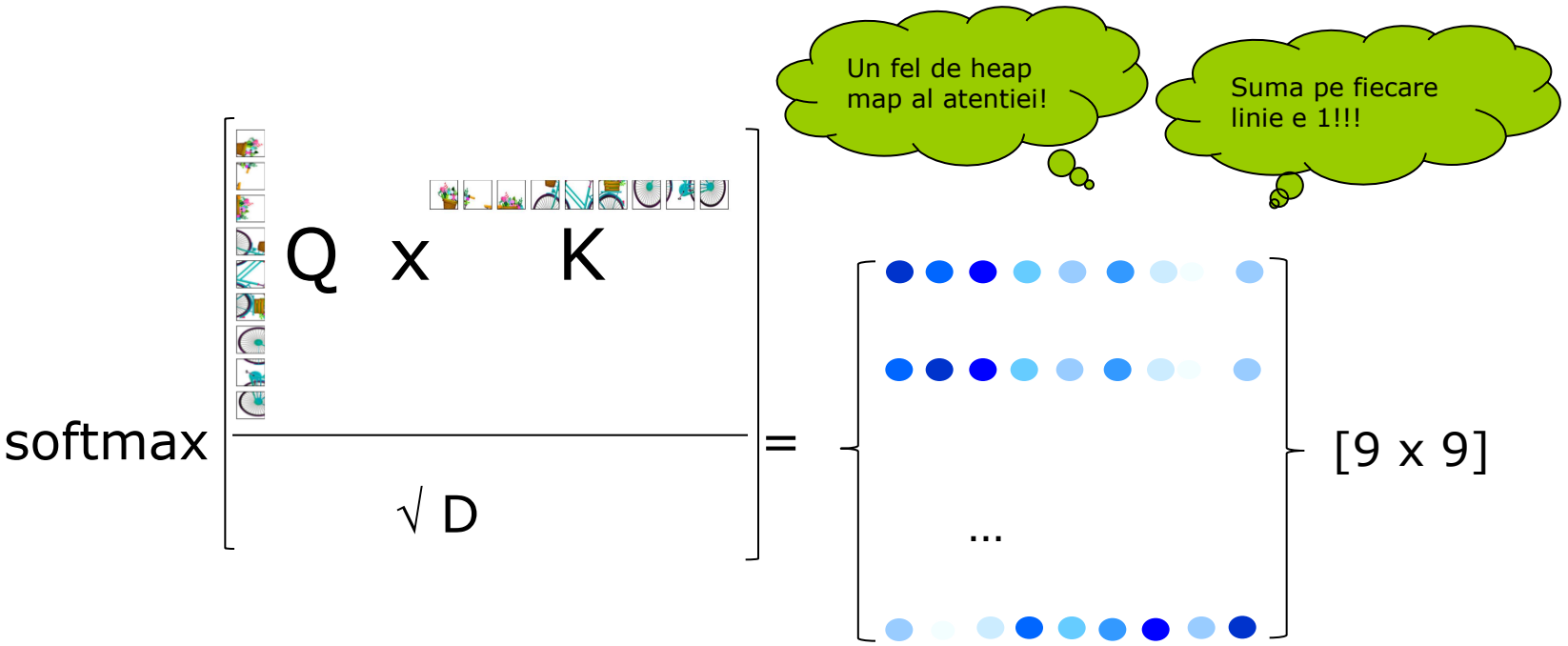


Mecanismul de “atentie” (attention)

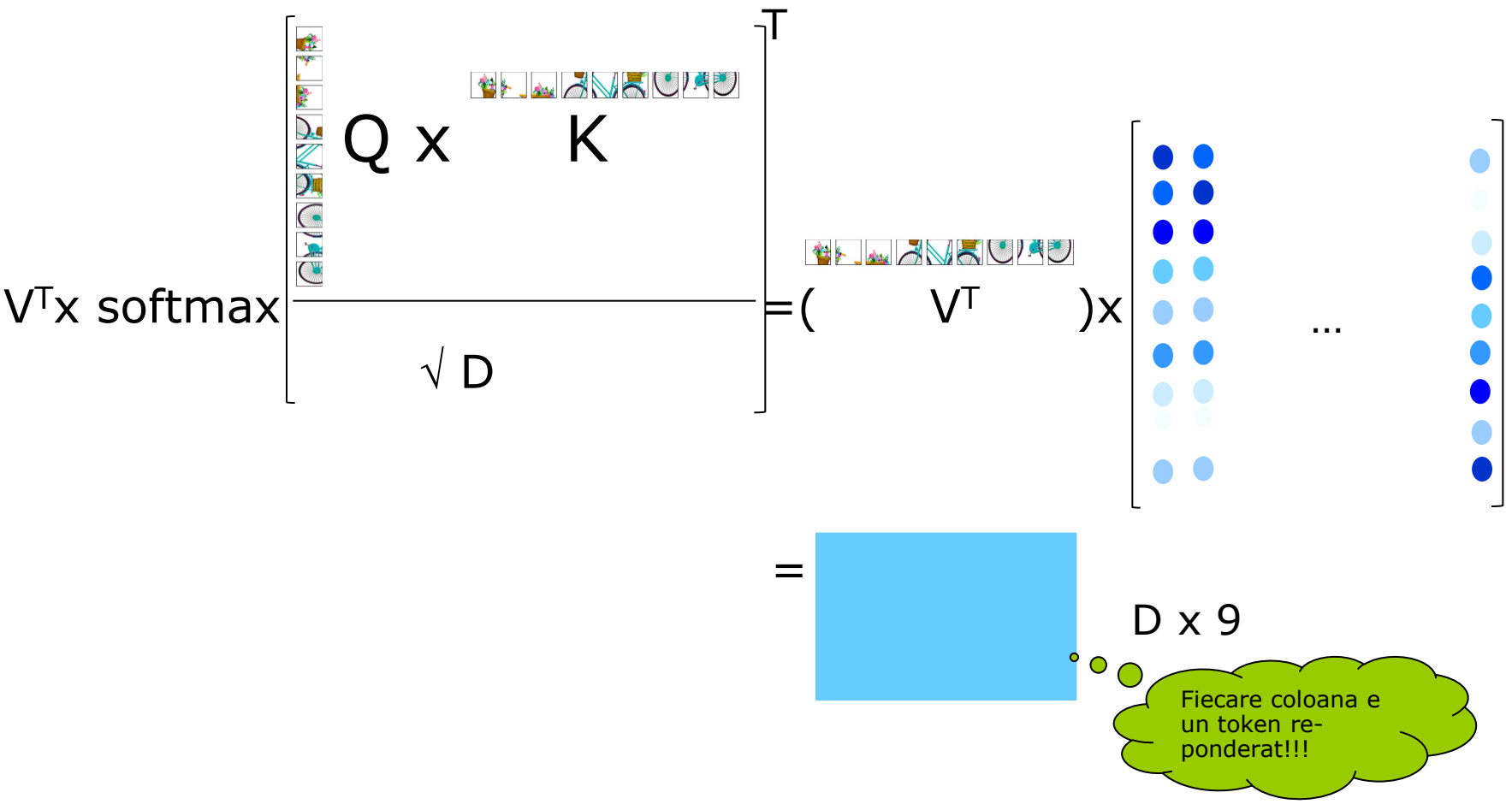




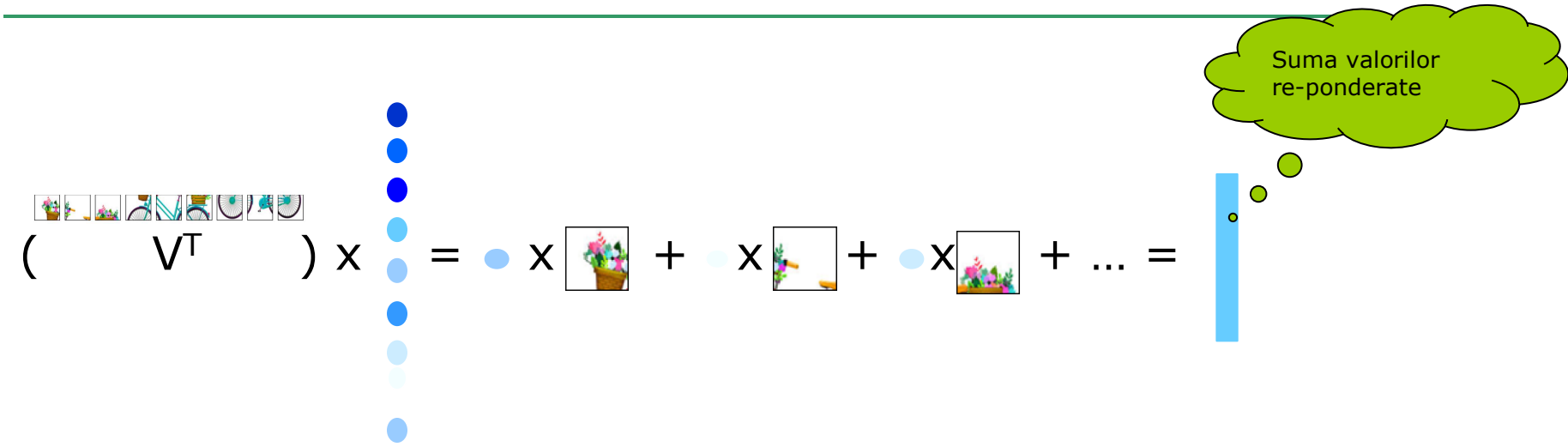
Mecanismul de “atentie” (attention)



Mecanismul de “atentie” (attention)

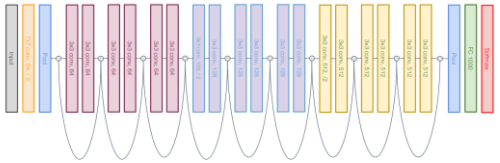


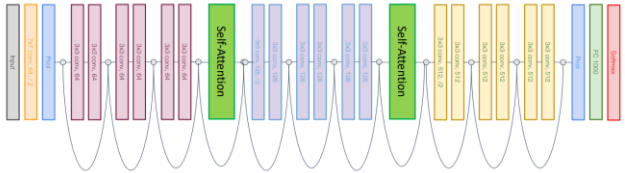
Mecanismul de “atentie” (attention)



Cum se poate integra mecanismul de atentie intr-o retea neuronala pentru procesarea imaginilor?

- V1: Integrarea atentiei intr-o CNN – de ex ResNet[1]
  - Cum?
    - Straturi suplimentare
  - Pro
    - Integrare facila
  - Contra
    - Modelul (arhitectura) e dominate de convolutii

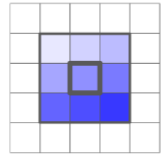


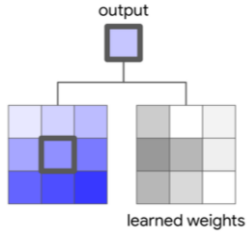


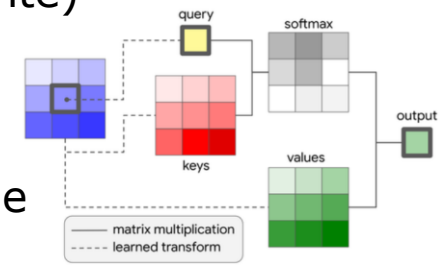
Wang, X., Girshick, R., Gupta, A., & He, K. (2018). Non-local neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 7794-7803) link

Cum se poate integra mecanismul de atentie intr-o retea neuronala pentru procesarea imaginilor?

- V2: Inlocuirea convolutiei cu o "atentie locala" ("local relation")
  - Cum?
    - Convolutia = produs scalar intre filtru si patch
    - Local attention
      - Centrul patch-ului -> query (vector cu D elemente)
      - Fiecare element din patch ->
        - Keys (vector R x R x D)
        - Values (vector R x R x C')
      - Outputul e calculate cu ajutorul mecanismului de atentie
  - Pro
    - Numar mai mic de parametrii (in Hu et al.: ResNet-50 are 25.5×10^6 param, iar LR-Net-50 are 23.3×10^6 params)
  - Contra
    - Implementare dificila (multe detalii tricky)
    - Doar putin mai buna ca ResNet





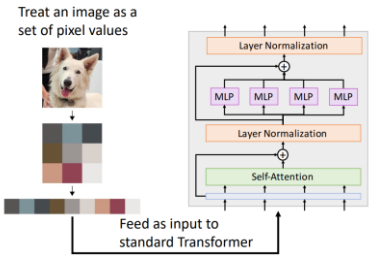


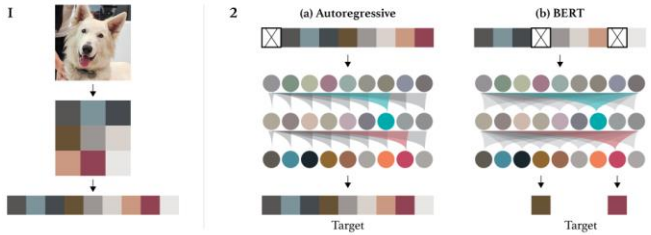
Hu, H., Zhang, Z., Xie, Z., & Lin, S. (2019). Local relation networks for image recognition. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 3464-3473) link

Ramachandran, P., Parmar, N., Vaswani, A., Bello, I., Levskaya, A., & Shlens, J. (2019). Stand-alone self-attention in vision models. *Advances in neural information processing systems*, *32* link

Cum se poate integra mecanismul de atentie intr-o retea neuronala pentru procesarea imaginilor?

- V3: transformer aplicat direct pe pixeli
  - Cum?
    - Input-ul pt transformer este imaginea (redimensionata) si aplatizata
- Pro
  - Simplu (conceptual)
- Contra
  - O imagine de dimensiune n x n necesita n^4 elemente in fiecare matrice de atentie -> multa memorie

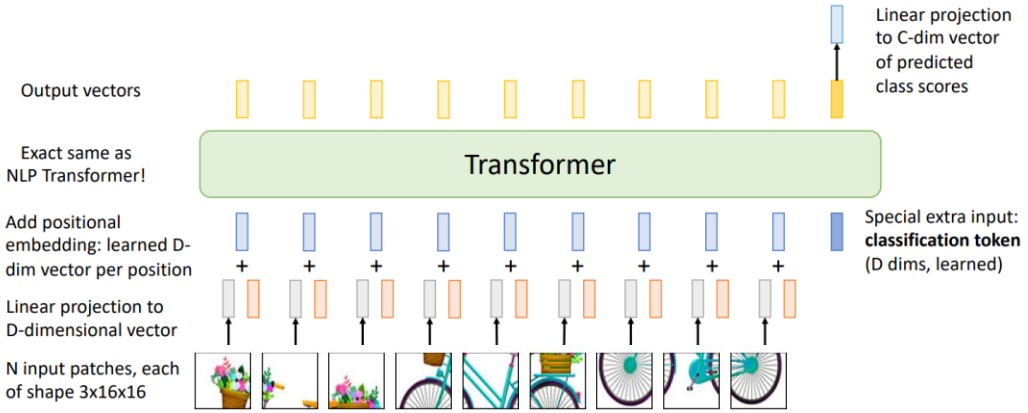




Chen, M., Radford, A., Child, R., Wu, J., Jun, H., Luan, D., & Sutskever, I. (2020, November). Generative pretraining from pixels. In *International conference on machine learning* (pp. 1691-1703). PMLR link

Cum se poate integra mecanismul de atentie intr-o retea neuronala pentru procesarea imaginilor?

- V4: transformer aplicat pe patch-uri
  - Cum?



  - Pro
    - Mai putine convolutii
  - Contra
    - Numar mare de parametrii
    - Nevoie de multe date de antrenament

Dosovitskiy, A. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*. link

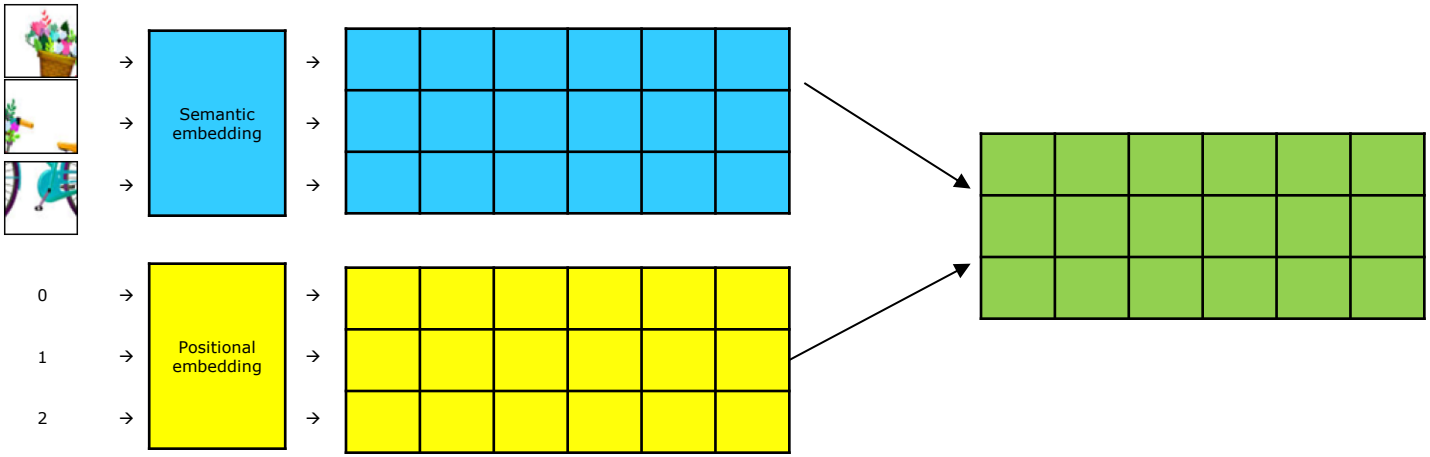
Positional embedding

- Patche-uri → Embeddings

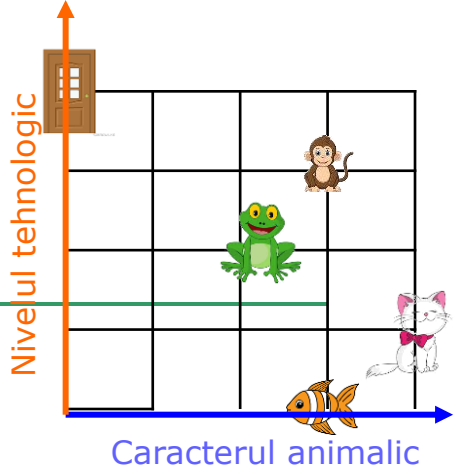
- Pentru o imagine cu k x k patch-uri, se obțin k x k embedding-uri (de lungime De)
  - Care țin cont de semantica patch-urilor
    - Linear Embeddings
      - descoperite / învățate
      - Cum? - prin ML (stratul linear al retelei neuronale)
  - Care țin cont de poziția (absolută sau relativă) a patch-urilor in imagine
    - Positional Emebeddings
      - calculate
      - cum? – prin formule :D

Positional embedding

- Exemplu
  - O imagine cu k x k patch-uri
  - Fiecărui patch i se asociază
    - Un embedding semantic SE de lungime d
    - Un embedding poziţional PE de lungime d
    - Embedding-ul final corespunzător patch-ului va fi suma celor 2 (WE + PE) → (kxk) x d valori



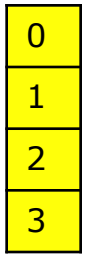
Positional embedding - text



- Ce valori conține PE?
  - Principii
    - Embedding-urile trebuie sa păstreze distanța originală între cuvinte
      - dist(Emb(câine),Emb(pisică)) < dist(Emb(câine),Emb(fereastră))
      - Dist(Emb($word_k$),Emb($word_{k+1}$))=Dist(Emb($word_p$),Emb($word_{p+1}$))
    - Ortogonalitate - Embedding-urile cuvintelor ne-conectate să fie perpendiculare
      - Emb(ușă) $\perp$ Emb(pește)
      - Similaritate (ușă, pește) = 0
    - Embedding-uri independente de lungimea propoziției
      - $\rightarrow$ putere de generalizare (train *vs*. test)
      - $\rightarrow$ *bounded*
  - Reprezentări / valori posibile
    - Poziția efectivă (1,2,3,4)
    - Poziția în reprezentare 1-hot encoding
    - Poziția în reprezentare binară
    - Poziția în reprezentare polară

Positional embedding – images
Position inside an image

- Exemplu
  - O imagine cu 4 patch-uri și d = 1
  - PE(patch) = poziția patch-ului

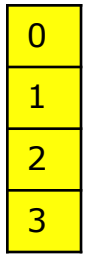


Dist	p0	p1	p2	p3
p0	0	1	2	3
p1	1	0	1	2
p2	2	1	0	1
p3	3	2	1	0

sim(PE(pi), PE(pj))!=0, orice i,j, i!= j

Positional embedding – images
Normalised position inside an image

- Exemplu
  - O imagine cu 4 patch-uri și d = 1
  - PE(patch) = poziția patch-ului



  - Problema: al k-lea patch într-o imagine cu n patch-uri are alt embedding decât al k-lea patch într-o propoziție cu m patch-uri

Positional embedding – images
1-hot encoding

0	0	0	0	0	1
0	0	0	0	1	0
0	0	0	1	0	0
0	0	1	0	0	0

- Exemplu
  - O imagine cu 4 patch-uri și d = 6
  - PE(patch) = reprezentarea 1-hot a poziției patch-ului

Dist	p0	p1	p2	p3
p0	0	1.4	1.4	1.4
p1	1.4	0	1.4	1.4
p2	1.4	1.4	0	1.4
p3	1.4	1.4	1.4	0

dist(PE(p0), PE(p1))=dist([0,0,0,0,0,1],[0,0,0,1,0]) = $\sqrt{(1^2+1^2)}$=1.41
sim(PE(wi), PE(wj))=0, orice i,j, i != j

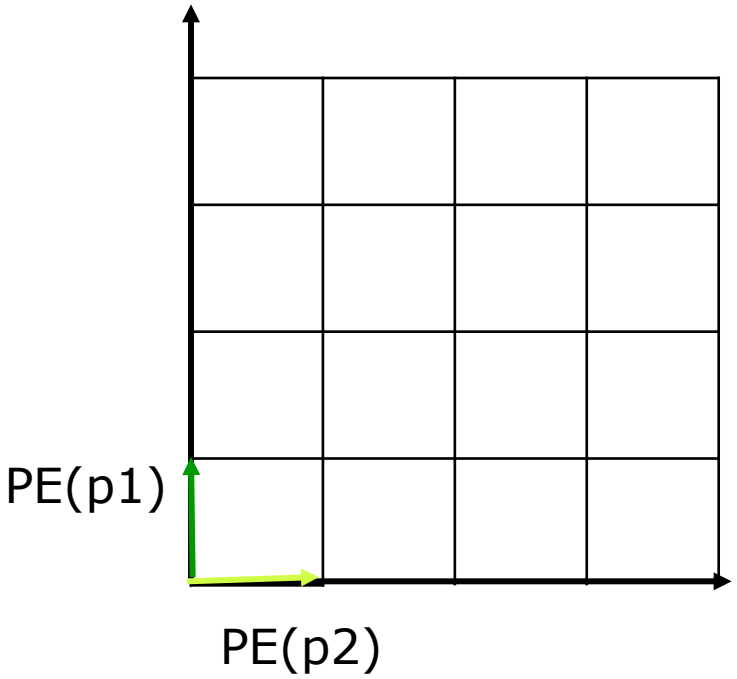
Positional embedding – images
Binary encoding

0	0	0	0	0	0
0	0	0	0	0	1
0	0	0	0	1	0
0	0	0	0	1	1

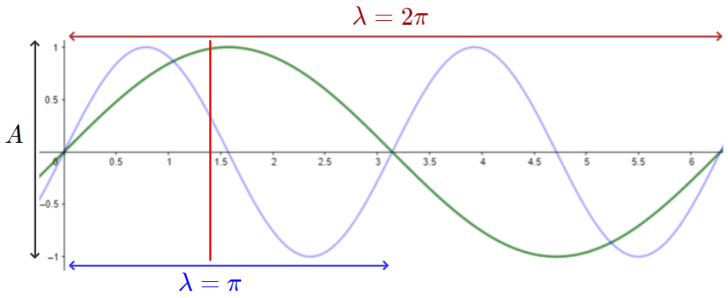
- Exemplu
  - O imagine cu 4 patch-uri și d = 6
  - PE(patch) = reprezentarea binara a poziției patch-ului

Dist	p0	p1	p2	p3
p0	0	1	1	√2
p1	1	0	√2	1
p2	1	√2	0	1
p3	√2	1	1	0

exista i, j a.î. sim(PE(pi), PE(pj))=0



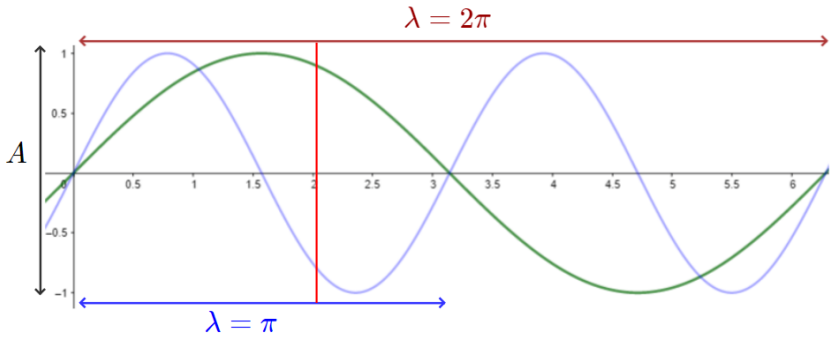
Positional embedding – images
Sin-based encoding



- Exemplu
  - O imagine cu 4 patch-uri și d = 6
  - PE(patch) = valorile functiei sin pentru diferite argumente (frecvențe sau lungimi de undă)
    - E.g. Sin(2 π pos / λ_i), i =0,1,2,..,d-1

pos		$\lambda = \pi$	$\lambda = 2\pi$	$\lambda = 3\pi$	$\lambda = 4\pi$	$\lambda = 5\pi$	$\lambda = 6\pi$
0	→	sin(2*0)	sin(0)	sin(2/3*0)	sin(2/4*0)	sin(2/5*0)	sin(2/6*0)
1	→	sin(2*1)	sin(1)	sin(2/3*1)	sin(2/4*1)	sin(2/5*1)	sin(2/6*1)
2	→	sin(2*2)	sin(2)	sin(2/3*2)	sin(2/4*2)	sin(2/5*2)	sin(2/6*2)
3	→	sin(2*3)	sin(3)	sin(2/3*3)	sin(2/4*3)	sin(2/5*3)	sin(2/6*3)
4	→	sin(2*4)	sin(4)	sin(2/3*4)	sin(2/4*4)	sin(2/5*4)	sin(2/6*4)

Positional embedding – images
Sin-based encoding

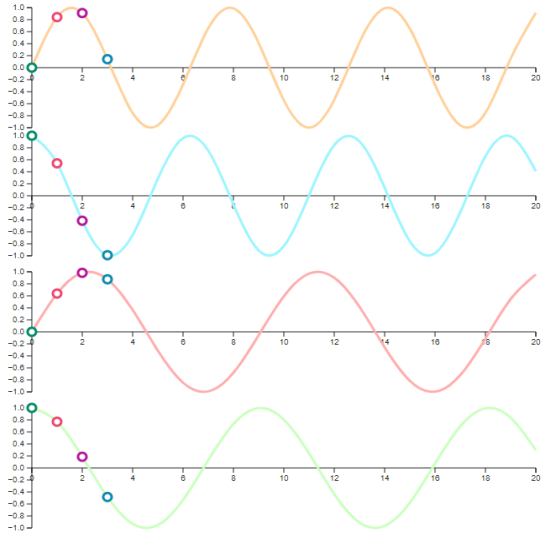


- Exemplu
  - O imagine cu 4 patch-uri și d = 6
  - PE(patch) = valorile functiei sin pentru diferite argumente (frecvențe sau lungimi de undă)
    - E.g. $Sin(2\pi pos / \lambda_j)$, $j=0,1,2,...,d-1$

pos		$\lambda=\pi$	$\lambda=2\pi$	$\lambda=3\pi$	$\lambda=4\pi$	$\lambda=5\pi$	$\lambda=6\pi$
0	→	sin(0)	sin(0)	sin(0)	sin(0)	sin(0)	sin(0)
1	→	sin(2*1)	sin(1)	sin(2/3*1)	sin(2/4*1)	sin(2/5*1)	sin(2/6*1)
2	→	sin(2*2)	sin(2)	sin(2/3*2)	sin(2/4*2)	sin(2/5*2)	sin(2/6*2)
3	→	sin(2*3)	sin(3)	sin(2/3*3)	sin(2/4*3)	sin(2/5*3)	sin(2/6*3)
4	→	sin(2*4)	sin(4)	sin(2/3*4)	sin(2/4*4)	sin(2/5*4)	sin(2/6*4)

sim(PE(p_0),PE(p_k))=0!!!

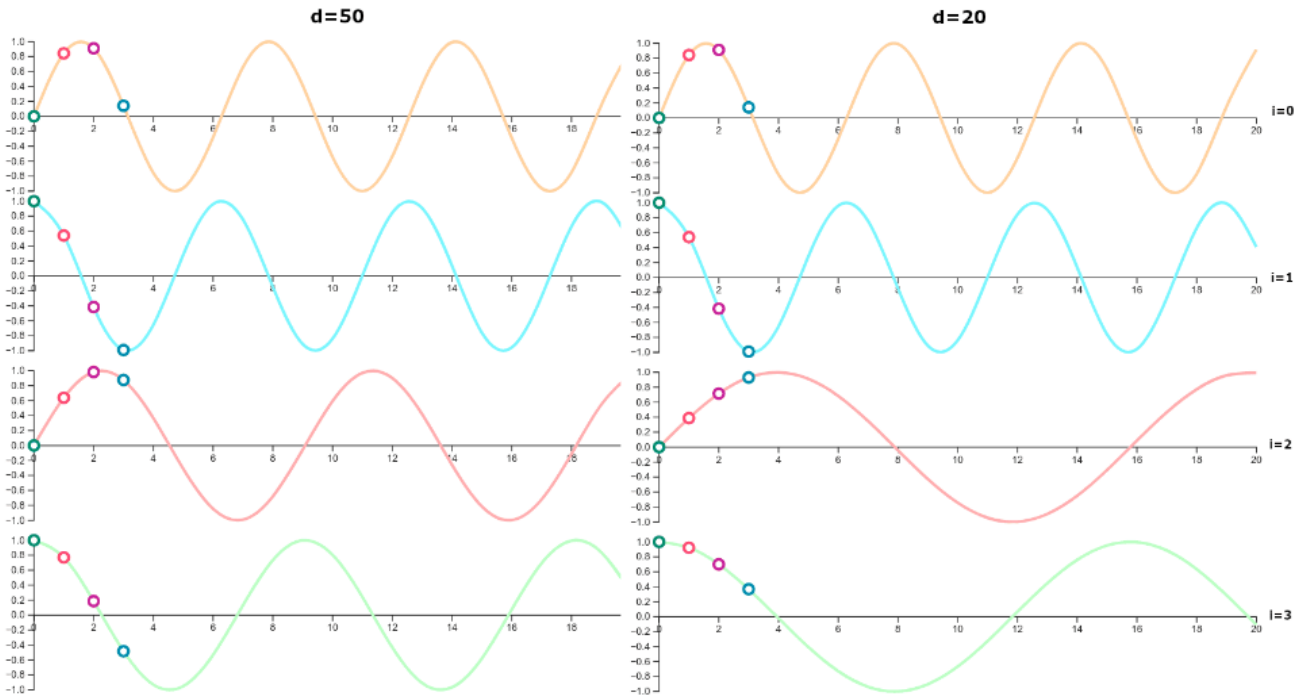
Positional embedding – in Sin&cos-based encoding



- Exemplu
  - O imagine cu 4 patch-uri și d = 6
  - PE(patch) = valorile functiei sin pentru diferite argumente (frecvențe sau lungimi de undă)
    - E.g. (sin(pos /$10000^{2i/d}$), cos(pos /$10000^{2i/d}$)), i =0,1,2,..,d-1

pos		i=0	i=1	i=2	i=3	i=4	i=5
0	→	sin(0)	cos(0)	sin(0)	cos(0)	sin(0)	cos(0)
1	→	sin(1)	cos(1)	sin(1/$10000^{4/6}$)	cos(1/$10000^{4/6}$)	sin(1/$10000^{8/6}$)	cos(1/$10000^{8/6}$)
2	→	sin(2)	cos(2)	sin(2/$10000^{4/6}$)	cos(2/$10000^{4/6}$)	sin(2/$10000^{8/6}$)	cos(2/$10000^{8/6}$)
3	→	sin(3)	cos(3)	sin(3/$10000^{4/6}$)	cos(3/$10000^{4/6}$)	sin(3/$10000^{8/6}$)	cos(3/$10000^{8/6}$)
4	→	sin(4)	cos(4)	sin(4/$10000^{4/6}$)	cos(4/$10000^{4/6}$)	sin(4/$10000^{8/6}$)	cos(4/$10000^{8/6}$)

Positional embedding – images
Sin&cos-based encoding



Cum se poate imbunatati performanta unui ViT?

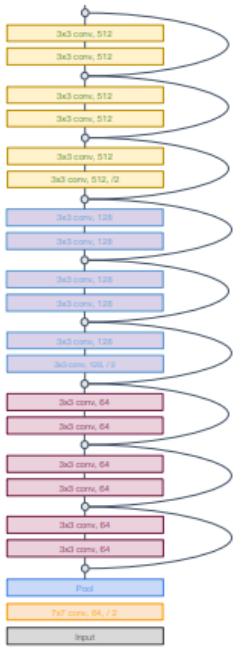
ViT vs CNN

Stage 3:
256 x 14 x 14

Stage 2:
128 x 28 x 28

Stage 1:
64 x 56 x 56

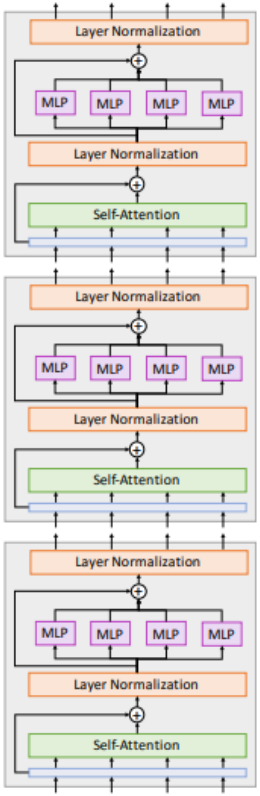
Input:
3 x 224 x 224



In most CNNs (including ResNets), **decrease** resolution and **increase** channels as you go deeper in the network (Hierarchical architecture)

Useful since objects in images can occur at various scales

In a ViT, all blocks have same resolution and number of channels (Isotropic architecture)



3rd block:
768 x 14 x 14

2nd block:
768 x 14 x 14

1st block:
768 x 14 x 14

Input:
3 x 224 x 224

Cum se poate imbunatati performanta unui ViT?

Input:

- A matrix of n x m values (e.g. a grayscale image of n x m pixels)

Layers:

- 1 convolution with F filters of k x k (stride 1)
  - Output: a matrix of (n -1) x (m – 2) x F values
    - #operations: [(n – 2) * (m – 2) * (k * k)] * F
    - #parameters: (k * k + 1) * F
- 1 pooling of p x p
  - Output: a matrix of (n / p) x (m / p) values
    - #operations: (n / p) * (m / p) * (p * p – 1)
    - #parameters: 0
- 1 fully connected layer
  - #input neurons = n * m
  - #output neurons = c
    - #operations: (n * m) * c
    - #parameters: (n * m) * c + c
- Transformer
  - Patching (p x p patches => L = n * m / (p * p) patches)) and Liniar transformation into a space of dimension d
    - Output: a matrix L x d
      - #operations: L * [(p * p) * d]
      - #parameters: [L * (p * p)] * d
  - Attention for an input of L x d values
    - Output: a matrix of L x d_{model} values
      - #operations:
        - Projections (Q, K, V): 3 * L * d * d_{model}
        - QK: L *L * d_{model}
        - Softmax: L * L
        - Weights: L * L * d_{model}
      - #parameters: 3 * (d * d_{model} * d_{model})
  - FeedForward layer:
    - Input: a matrix of L x d_{model} values
    - Output: a matrix of L x out values
      - #operations: (L * L) * (d_{model} * d_{out} + d_{out})
      - #parameters: d_{model} * d_{out} + d_{out}

M I R P R

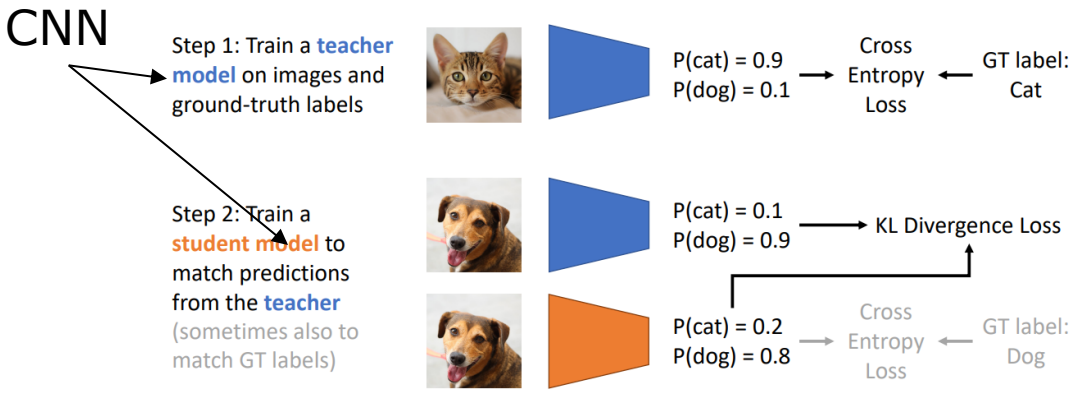
Cum se poate imbunatati performanta unui ViT?

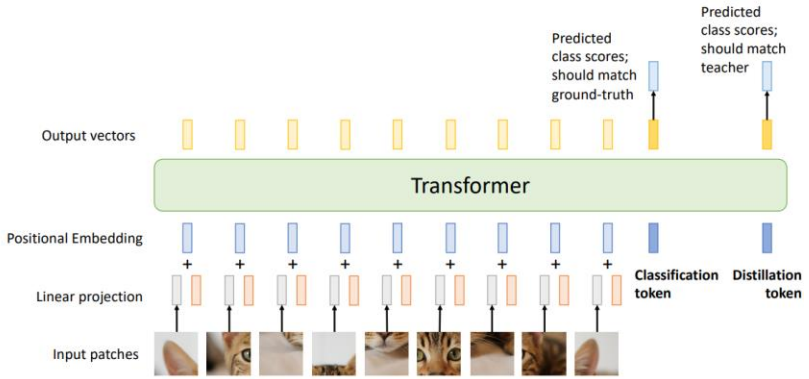
- Regularizare
  - Weight decay, stochastic depth, dropout
- Data augmentation
  - MixUp, RandAugment

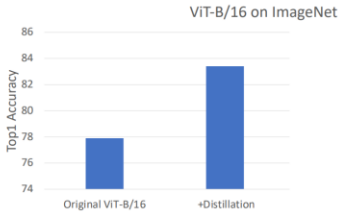
Steiner, A., Kolesnikov, A., Zhai, X., Wightman, R., Uszkoreit, J., & Beyer, L. (2021). How to train your vit? data, augmentation, and regularization in vision transformers. *arXiv preprint arXiv:2106.10270*. link

Cum se poate imbunatati performanta unui ViT?

- Distillation







Hinton, G. (2015). Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* link

Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., & Jégou, H. (2021, July). Training data-efficient image transformers & distillation through attention. In *International conference on machine learning* (pp. 10347-10357). PMLR link

Cum se poate imbunatati performanta unui ViT?

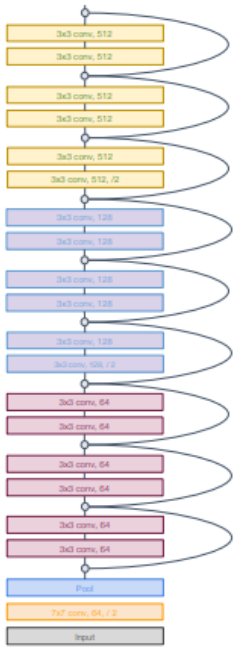
ViT vs CNN

Stage 3:
256 x 14 x 14

Stage 2:
128 x 28 x 28

Stage 1:
64 x 56 x 56

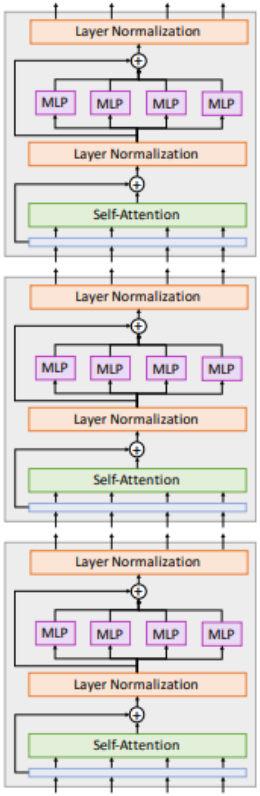
Input:
3 x 224 x 224



In most CNNs (including ResNets), **decrease** resolution and **increase** channels as you go deeper in the network (Hierarchical architecture)

Useful since objects in images can occur at various scales

In a ViT, all blocks have same resolution and number of channels (Isotropic architecture)



3rd block:
768 x 14 x 14

2nd block:
768 x 14 x 14

1st block:
768 x 14 x 14

Input:
3 x 224 x 224

More details

- https://github.com/google-research/vision_transformer
- https://huggingface.co/docs/transformers/model_doc/vit